



Politécnico Gracolombiano - Facultad de Ingeniería, Diseño e Innovación

Cristhian Felipe Ramírez Chaves
Raquel Angelica Velandia Cubillos

Proyecto de investigación de Computación de Alto Desempeño (HPC)

Evaluación del rendimiento paralelo en el procesamiento de grandes volúmenes de datos utilizando DuckDB: comparación experimental entre formatos Apache Parquet y CSV en un entorno HPC

Semillero de investigación

Prof. Víctor Augusto Pedraza Bonilla

Bogotá D.C.

Tabla de Contenido

1.	Planteamiento del problema	5
2.	Pregunta de investigación.....	5
3.	Hipótesis	6
4.	Objetivos	6
4.1.	Objetivo general:	6
4.2.	Objetivos específicos:	6
5.	Metodología experimental.....	7
5.1.	Entorno de pruebas:	8
5.2.	Dataset:	9
5.3.	Dataset sugerido:	9
5.4.	Selección del formato de almacenamiento.....	10
5.5.	Arquitectura del sistema propuesto:	12
6.	Implementación del procesamiento en Python:	13
6.1.	Manejo de errores y excepciones.....	15
6.2.	Configuración del paralelismo:	16
7.	Operaciones analíticas de prueba:	17
7.1.	Conteo total de registros (COUNT)	17
7.2.	Suma total de pasajeros (SUM)	17
7.3.	Promedio de distancia recorrida (AVG)	18
7.4.	Agrupación por cantidad de pasajeros (GROUP BY).....	18
7.5.	Filtrado y agrupamiento analítico (WHERE + GROUP BY).....	18
7.6.	Consulta 1 - Conteo total de registros:	19
7.7.	Consulta 2 - Filtrado de registros:	20
7.8.	Consulta 3 - Agregación por categoría:	21
7.9.	Consulta 4 - Ordenamiento de registros:.....	22
7.10.	Consulta 5 - Agregación temporal:.....	23
8.	Métricas de evaluación	24
8.1.	Tiempo de ejecución	25
8.2.	Speedup	25
8.3.	Eficiencia paralela	25
8.4.	Registros válidos e inválidos	26
8.5.	Excepciones capturadas	26
8.6.	Escalabilidad del procesamiento	27

9.	Procedimiento experimental	27
9.1.	Preparación del dataset	27
9.2.	Configuración del entorno experimental.....	27
9.3.	Ejecución del escenario Apache Parquet.....	28
9.4.	Generación de archivos CSV con errores controlados.....	28
9.5.	Procesamiento de archivos CSV	29
9.6.	Registro de métricas experimentales	29
9.7.	Análisis comparativo de resultados	29
10.	Presentación y análisis de resultados.....	29
10.1.	Resultados experimentales utilizando Apache Parquet	30
10.3.	Resultados de speedup	32
10.4.	Análisis del comportamiento de las consultas	34
11.	Resultados experimentales utilizando archivos CSV con errores controlados	35
11.1.	Resultados de validación y manejo de errores	36
11.2.	Resultados de tiempo promedio de ejecución	39
11.3.	Consulta Q2 – Suma total de pasajeros	40
11.4.	Consulta Q3 – Promedio de distancia recorrida	41
11.4.1.	Consulta Q4 – Agrupación por cantidad de pasajeros.....	42
11.4.2.	Consulta Q5 – Filtrado y agrupamiento combinado	43
11.5.	Resultados de speedup	44
11.5.1.	Consulta Q1 – Conteo total de registros.....	45
11.5.2.	Consulta Q2 – Suma total de pasajeros	45
11.5.3.	Consulta Q3 – Promedio de distancia recorrida	46
11.5.4.	Consulta Q4 – Agrupación por cantidad de pasajeros.....	47
11.5.5.	Consulta Q5 – Filtrado y agrupamiento combinado	48
11.6.	Resultados de eficiencia paralela	49
11.6.1.	Consulta Q1 – Conteo total de registros.....	49
11.6.2.	Consulta Q2 – Suma total de pasajeros	50
11.6.3.	Consulta Q3 – Promedio de distancia recorrida	51
11.6.4.	Consulta Q4 – Agrupación por cantidad de pasajeros.....	52
11.6.5.	Consulta Q5 – Filtrado y agrupamiento combinado	53
11.7.	Comparación entre Apache Parquet y CSV.....	54
11.7.1.	Comparación de tiempo de ejecución	54
11.7.2.	Comparación de speedup	55

12.	Discusión comparativa general	56
13.	Discusión general de resultados	57
14.	Limitaciones del experimento y consideraciones:	58
14.1.	Limitaciones del hardware utilizado	58
14.2.	Limitaciones del procesamiento distribuido	59
14.3.	Diferencias entre motores de procesamiento	59
14.4.	Impacto de caché y estado del sistema operativo	59
14.5.	Limitaciones relacionadas con los datos generados.....	60
14.6.	Sobrecarga del paralelismo	60
14.7.	Alcance experimental del estudio	60
15.	Conclusiones	61
16.	Trabajo futuro y propuestas de ampliación	62
17.	Bibliografía	64

1. Planteamiento del problema

El crecimiento acelerado de los volúmenes de datos generados en entornos digitales ha incrementado la necesidad de desarrollar soluciones capaces de procesar grandes cantidades de información de manera eficiente y robusta. Actualmente, múltiples sectores como el transporte, las plataformas financieras, las telecomunicaciones y los sistemas inteligentes generan millones de registros diariamente, lo que exige mecanismos de procesamiento optimizados capaces de aprovechar arquitecturas multinúcleo y técnicas de paralelismo.

En el contexto de la Computación de Alto Desempeño (HPC), el procesamiento paralelo permite reducir significativamente los tiempos de ejecución en tareas analíticas sobre grandes volúmenes de datos. Herramientas modernas como DuckDB han sido diseñadas específicamente para ejecutar operaciones analíticas de alto rendimiento utilizando múltiples hilos de procesamiento y formatos optimizados como Apache Parquet, el cual ofrece almacenamiento columnar, compresión eficiente y lectura selectiva de datos.

Sin embargo, en escenarios reales de procesamiento de información no todos los datos se encuentran organizados bajo formatos optimizados o completamente estructurados. En muchos entornos prácticos es común trabajar con archivos CSV que pueden contener registros incompletos, valores nulos, errores de formato o inconsistencias de datos. Estas situaciones obligan a implementar mecanismos de validación, manejo de excepciones y tolerancia a fallos durante el procesamiento de la información, introduciendo un costo computacional adicional que puede impactar el rendimiento del sistema.

A partir de este contexto, el presente proyecto propone una evaluación experimental del procesamiento paralelo de grandes volúmenes de datos utilizando DuckDB sobre dos escenarios diferentes: un entorno optimizado basado en archivos Apache Parquet libres de errores y un entorno basado en archivos CSV con errores controlados y mecanismos de manejo de excepciones. El objetivo es analizar no solo el impacto del paralelismo multinúcleo en el rendimiento del procesamiento analítico, sino también evaluar el comportamiento del sistema frente a escenarios más cercanos a condiciones reales de ingeniería de datos.

La investigación incluye la ejecución de consultas analíticas sobre datasets masivos de transporte urbano, el uso de diferentes configuraciones de hilos de CPU y el análisis de métricas como tiempo de ejecución, speedup y eficiencia paralela. Adicionalmente, se evaluará el impacto del manejo de excepciones y validación de datos sobre el rendimiento del procesamiento en archivos CSV con errores intencionales.

2. Pregunta de investigación

¿Cuáles son las diferencias de rendimiento, escalabilidad y robustez en el procesamiento paralelo de grandes volúmenes de datos utilizando DuckDB sobre

archivos Apache Parquet y archivos CSV con errores y manejo de excepciones en un entorno HPC?

3. Hipótesis

- El uso de técnicas de paralelismo multinúcleo permitirá reducir significativamente el tiempo de ejecución en el procesamiento de grandes volúmenes de datos, especialmente en operaciones analíticas ejecutadas sobre archivos Apache Parquet utilizando DuckDB.
- El formato Apache Parquet presentará un mejor rendimiento y escalabilidad en comparación con archivos CSV, debido a su estructura de almacenamiento columnar y a sus mecanismos de optimización para procesamiento analítico paralelo.
- El procesamiento de archivos CSV con errores controlados requerirá mecanismos adicionales de validación y manejo de excepciones, lo que generará un incremento en el tiempo de ejecución y una disminución de la eficiencia del procesamiento paralelo.
- Las consultas analíticas complejas que involucren filtrados, agrupamientos y agregaciones presentarán un mayor aprovechamiento del paralelismo en comparación con operaciones simples como conteos directos de registros.
- La implementación de mecanismos de manejo de excepciones permitirá procesar archivos CSV con registros inconsistentes sin detener completamente la ejecución del sistema, mejorando la robustez del procesamiento frente a escenarios reales de ingeniería de datos.

4. Objetivos

4.1. Objetivo general:

Evaluar el impacto del paralelismo multinúcleo en el procesamiento de grandes volúmenes de datos mediante el uso de DuckDB sobre archivos Apache Parquet y archivos CSV con errores controlados, analizando métricas de rendimiento, escalabilidad y robustez en un entorno de Computación de Alto Desempeño (HPC).

4.2. Objetivos específicos:

- i. Identificar herramientas y tecnologías orientadas al procesamiento paralelo de grandes volúmenes de datos en entornos HPC.
- ii. Implementar consultas analíticas utilizando DuckDB sobre datasets masivos almacenados en formato Apache Parquet.

- iii. Utilizar el motor analítico DuckDB para ejecutar consultas sobre el mismo conjunto de datos.
- iv. Diseñar una aplicación de escritorio que permita ejecutar las pruebas experimentales y controlar el número de hilos utilizados en el procesamiento.
- v. Configurar diferentes niveles de paralelismo mediante múltiples hilos de CPU para evaluar el comportamiento escalable del procesamiento analítico.
- vi. Medir métricas de rendimiento como tiempo de ejecución, speedup y eficiencia paralela durante la ejecución de operaciones analíticas sobre archivos Parquet.
- vii. Generar archivos CSV con errores controlados a partir de datasets estructurados para simular escenarios reales de procesamiento de datos.
- viii. Implementar mecanismos de validación de datos y manejo de excepciones durante el procesamiento de archivos CSV con registros inconsistentes.
- ix. Comparar el comportamiento del procesamiento analítico entre archivos Apache Parquet optimizados y archivos CSV con errores y manejo de excepciones.
- x. Analizar el impacto del manejo de errores y validación de datos sobre el rendimiento y la eficiencia del procesamiento paralelo.

5. Metodología experimental

El presente estudio se desarrolla mediante un enfoque experimental orientado a evaluar el impacto del paralelismo multinúcleo en el procesamiento de grandes volúmenes de datos. Para ello, se comparan dos escenarios de procesamiento sobre el dataset NYC Yellow Taxi Trip Records:

- Procesamiento optimizado sobre archivos Apache Parquet libres de errores, utilizando DuckDB como motor analítico.
- Procesamiento sobre archivos CSV con errores controlados, incorporando mecanismos de validación de datos y manejo de excepciones.

El primer escenario representa un entorno optimizado de análisis de datos, en el cual los archivos se encuentran estructurados, tipados y almacenados en un formato columnar eficiente. Este escenario permite medir el rendimiento de DuckDB en operaciones analíticas ejecutadas sobre grandes volúmenes de datos, aprovechando múltiples hilos de CPU.

El segundo escenario representa una situación más cercana a contextos reales de ingeniería de datos, en los cuales los archivos CSV pueden contener registros incompletos, valores nulos, errores de formato, datos inconsistentes o valores fuera de rango. En este caso, el procesamiento requiere rutinas adicionales de validación, captura de excepciones y registro de errores, con el fin de evitar que registros defectuosos interrumpan la ejecución completa del sistema.

Ambos escenarios se ejecutan utilizando diferentes configuraciones de paralelismo, con el propósito de analizar el comportamiento del sistema frente al aumento del número de hilos de procesamiento. Las pruebas permiten comparar el impacto del formato de almacenamiento, la calidad de los datos y el manejo de excepciones sobre métricas como tiempo de ejecución, speedup, eficiencia paralela y cantidad de errores detectados. La metodología experimental se estructura en las siguientes etapas:

- Preparación del dataset base.
- Selección del formato Apache Parquet como escenario optimizado.
- Generación de archivos CSV con errores controlados a partir del dataset original.
- Configuración del entorno experimental y del número de hilos de CPU.
- Ejecución de consultas analíticas sobre archivos Parquet mediante DuckDB.
- Procesamiento de archivos CSV con validación de datos y manejo de excepciones.
- Registro de métricas de rendimiento, escalabilidad y robustez.
- Análisis comparativo entre el escenario Parquet optimizado y el escenario CSV con errores.

Este enfoque permite evaluar no solo el rendimiento del procesamiento paralelo sobre datos optimizados, sino también el costo computacional asociado al manejo de errores en datos menos estructurados. De esta manera, el experimento integra dos dimensiones complementarias: la eficiencia del procesamiento analítico y la robustez frente a inconsistencias en los datos.

5.1. Entorno de pruebas: El experimento se ejecutará en un sistema con procesador multinúcleo capaz de ejecutar múltiples hilos de procesamiento simultáneamente. Se propone el siguiente sistema:

Tabla 1 – Equipo seleccionado

Componente	Especificación
Equipo	ASUS ROG Strix
CPU	AMD Ryzen 9 5900HX
Núcleos	8
Hilos	16
GPU	NVIDIA RTX 3070 Laptop

Componente	Especificación
VRAM	8 GB
RAM	16 GB
Almacenamiento	Samsung SSD 990 EVO Plus 2Tb
Sistema operativo	Windows / Linux

5.2. Dataset: Para la ejecución del experimento se utilizará un dataset de gran tamaño compuesto únicamente por datos estructurados. El dataset seleccionado deberá cumplir con las siguientes características:

- Grande (≥ 15 GB) para que el paralelismo realmente tenga impacto.
- Datos tabulares (CSV / JSON).
- Millones de registros.
- Columnas que permitan agregaciones, filtros y joins.
- Datos reales (no sintéticos) para darle más valor académico al proyecto.

Estos requisitos permiten asegurar que el volumen de información sea lo suficientemente grande para que el paralelismo tenga un impacto observable en el rendimiento del procesamiento.

5.3. Dataset sugerido: NYC Taxi Trip Dataset:

- Tamaño: 20 GB – 200 GB dependiendo de los años incluidos
- Formato: CSV, Parquet
- Registros: Más de 1 billón de viajes
- Columnas típicas: pickup_datetime, dropoff_datetime, passenger_count, trip_distance, pickup_longitude, pickup_latitude, dropoff_longitude, dropoff_latitude, fare_amount, tip_amount, payment_type
- Ventajas para HPC: Permite consultas pesadas como tipo de pago, tipo de vehículo, tipo de pago, análisis por tiempo, ubicación entre otros parámetros.

El NYC Taxi Trip Dataset (<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>), un conjunto de datos públicos publicado por la *New York City Taxi and Limousine Commission (TLC)* que contiene registros detallados de viajes realizados por taxis en la ciudad de Nueva York. Este dataset es ampliamente utilizado en investigaciones relacionadas con análisis de datos, procesamiento analítico y benchmarking de sistemas de bases de datos, debido a su gran volumen de registros y a la naturaleza estructurada de la información que contiene. Los registros incluyen información relevante sobre cada viaje, como:

- Fecha y hora de recogida
- Fecha y hora de finalización
- Distancia del viaje

- Número de pasajeros
- Tipo de pago
- Tarifa del viaje
- Ubicación de origen y destino

Estas características permiten realizar diversas operaciones analíticas, como filtrado, agregaciones, agrupamientos y análisis temporal de los datos. El dataset se encuentra dividido en múltiples archivos organizados por mes y por tipo de vehículo. Para este proyecto se utilizarán exclusivamente los registros correspondientes a Yellow Taxi Trip Records, ya que representan el conjunto de datos más utilizado en análisis de movilidad urbana y presentan una estructura consistente a lo largo de los diferentes periodos de tiempo. La utilización de múltiples archivos mensuales permite construir datasets de diferentes tamaños, lo cual resulta útil para evaluar el comportamiento del sistema frente a distintos volúmenes de información.

5.4. Selección del formato de almacenamiento

Los registros del dataset NYC Yellow Taxi Trip Records se encuentran disponibles en diferentes formatos de almacenamiento, entre los que se destacan Apache Parquet y CSV. Para el presente proyecto se decidió trabajar con ambos formatos debido a que representan escenarios diferentes dentro del procesamiento masivo de datos.

El formato Apache Parquet fue seleccionado como escenario optimizado de procesamiento analítico, debido a sus ventajas en rendimiento, almacenamiento columnar y aprovechamiento del paralelismo. Parquet es un formato diseñado para sistemas de análisis de datos a gran escala, permitiendo leer únicamente las columnas necesarias para ejecutar una consulta determinada. Esta característica reduce significativamente el volumen de datos leído desde el disco y mejora el rendimiento en operaciones analíticas que involucran agregaciones, filtrados y agrupamientos. Entre las principales ventajas del formato Parquet se destacan:

- Almacenamiento columnar optimizado para análisis de datos.
- Compresión eficiente que reduce el tamaño de los archivos.
- Lectura selectiva de columnas.
- Mejor aprovechamiento del paralelismo en arquitecturas multinúcleo.
- Mayor eficiencia en consultas analíticas sobre grandes volúmenes de información.

Estas características convierten a Parquet en uno de los formatos más utilizados en entornos de Big Data y motores analíticos modernos como DuckDB, Apache Spark y herramientas de procesamiento de datos en Python. Por otro lado, el formato CSV fue incorporado como escenario experimental orientado a simular condiciones más cercanas a contextos reales de ingeniería de datos. Aunque CSV presenta menores niveles de optimización para procesamiento analítico, continúa

siendo ampliamente utilizado debido a su simplicidad, compatibilidad y facilidad de intercambio entre sistemas.

Para este proyecto, los archivos CSV serán generados a partir del dataset original e incluirán errores controlados, tales como valores nulos, inconsistencias de formato, datos fuera de rango y registros incompletos. Este enfoque permitirá evaluar el impacto del manejo de excepciones, validación de datos y tolerancia a fallos sobre el rendimiento del procesamiento paralelo.

La utilización conjunta de ambos formatos permite comparar dos escenarios complementarios:

- Un entorno optimizado basado en archivos Parquet libres de errores.
- Un entorno orientado a robustez y validación utilizando archivos CSV con errores controlados.

De esta manera, el experimento no solo analiza el rendimiento del procesamiento paralelo en condiciones ideales, sino también el comportamiento del sistema frente a situaciones reales donde los datos pueden contener inconsistencias o errores de calidad.

Tabla 2 – Características del dataset

Característica	Valor
Dataset principal	NYC Yellow Taxi Trip Records
Fuente	NYC Taxi & Limousine Commission
URL oficial	https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Formatos utilizados	Apache Parquet y CSV
Escenario 1	Archivos Parquet optimizados y libres de errores
Escenario 2	Archivos CSV con errores controlados y manejo de excepciones
Periodo de datos	2018–2023
Tamaño estimado total	18–20 GB aproximadamente
Número aproximado de archivos	72 archivos Parquet + archivos CSV generados experimentalmente
Tipo de datos	Datos tabulares estructurados
Dominio del dataset	Transporte y movilidad urbana
Operaciones analíticas evaluadas	COUNT, SUM, AVG, GROUP BY y filtros analíticos
Variables analizadas	passenger_count, trip_distance, fare_amount, payment_type, pickup_datetime
Configuración experimental	Procesamiento multinúcleo utilizando múltiples hilos de CPU

Característica	Valor
Métricas registradas	Tiempo de ejecución, speedup, eficiencia paralela y manejo de errores

5.5.Arquitectura del sistema propuesto:

La arquitectura propuesta para el presente proyecto se encuentra orientada a evaluar el comportamiento del procesamiento paralelo de grandes volúmenes de datos bajo dos escenarios experimentales diferentes: un entorno optimizado utilizando archivos Apache Parquet y un entorno basado en archivos CSV con errores controlados. El sistema se compone de los siguientes componentes principales:

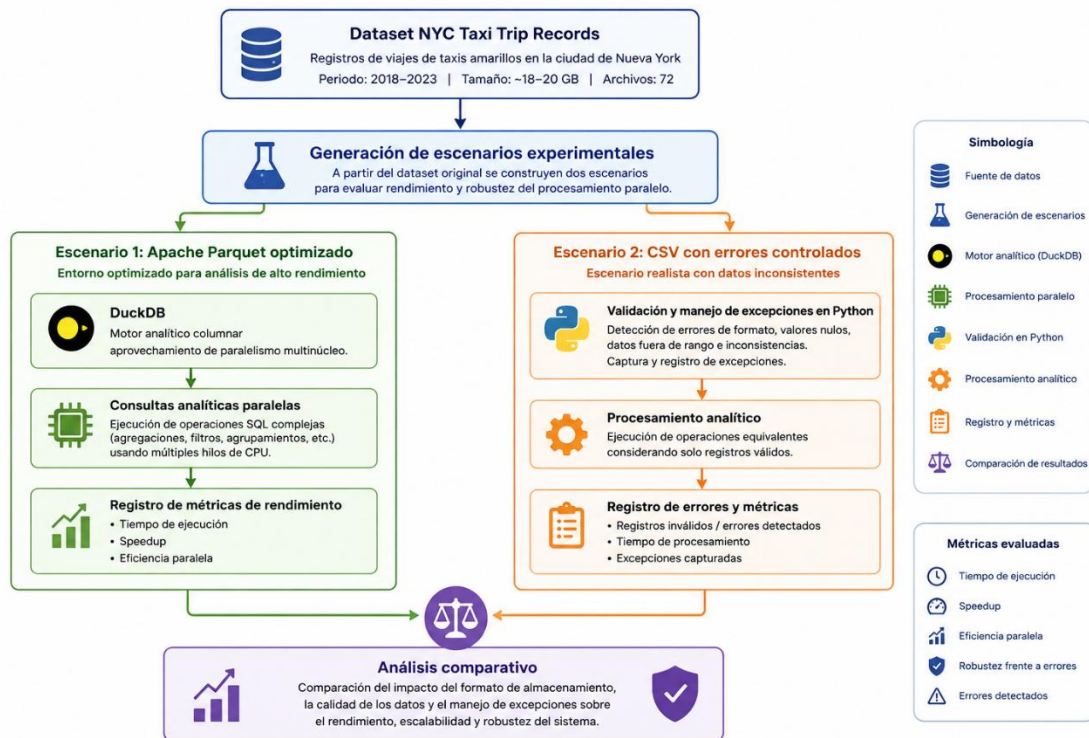
- Dataset base NYC Yellow Taxi Trip Records.
- Archivos Apache Parquet libres de errores.
- Archivos CSV generados con errores controlados.
- Motor analítico DuckDB.
- Módulo de validación y manejo de excepciones en Python.
- Sistema de ejecución de benchmarks y registro de métricas.

En el primer escenario experimental, los archivos Apache Parquet son procesados directamente mediante DuckDB utilizando consultas SQL analíticas y diferentes configuraciones de paralelismo. Este flujo permite evaluar el rendimiento del procesamiento multinúcleo sobre datos estructurados y optimizados para análisis de alto rendimiento.

En el segundo escenario experimental, los archivos CSV son generados a partir del dataset original e incluyen errores controlados, como valores nulos, inconsistencias de formato, registros incompletos y datos fuera de rango. Estos archivos son procesados mediante rutinas desarrolladas en Python que implementan validación de datos, captura de excepciones y registro de errores durante la ejecución.

Ambos escenarios ejecutan operaciones analíticas equivalentes con el objetivo de comparar el impacto del formato de almacenamiento, la calidad de los datos y el manejo de excepciones sobre el rendimiento del procesamiento paralelo.

La arquitectura general del sistema puede describirse de la siguiente manera:



Las métricas registradas durante ambos escenarios incluyen tiempo de ejecución, speedup, eficiencia paralela, cantidad de registros inválidos y comportamiento del sistema frente a errores de datos.

Este diseño permite comparar un entorno de procesamiento optimizado frente a un entorno más cercano a situaciones reales de ingeniería de datos, donde los archivos pueden contener inconsistencias que requieren validación y tolerancia a fallos.

6. Implementación del procesamiento en Python:

Como parte del proyecto se desarrolló un conjunto de rutinas en Python orientadas al procesamiento de archivos CSV generados a partir del dataset original NYC Yellow Taxi Trip Records. A diferencia del escenario optimizado basado en Apache Parquet, esta implementación se enfocó en simular condiciones más cercanas a entornos reales de ingeniería de datos, donde los archivos pueden contener inconsistencias, errores de formato y registros corruptos.

La implementación en Python cumple dos funciones principales dentro del experimento:

- Generación de archivos CSV con errores controlados.
- Procesamiento de datos con validación y manejo de excepciones.

Inicialmente, se desarrolló un proceso de conversión de archivos Apache Parquet hacia archivos CSV, incorporando errores aleatorios sobre diferentes columnas del dataset. Entre los errores introducidos se encuentran:

- Valores nulos.
- Datos fuera de rango.
- Registros incompletos.
- Valores de texto en columnas numéricas.
- Fechas inválidas.
- Inconsistencias de formato.

El objetivo de esta etapa fue construir un escenario experimental que permitiera evaluar el impacto del manejo de errores sobre el rendimiento del procesamiento paralelo.

Posteriormente, se implementó un módulo de procesamiento en Python encargado de leer los archivos CSV y ejecutar operaciones analíticas equivalentes a las utilizadas en el escenario Parquet. Durante este proceso se incorporaron mecanismos de validación de datos y captura de excepciones mediante estructuras try/except, evitando que registros defectuosos interrumpieran completamente la ejecución del sistema.

La implementación permite identificar registros inválidos, registrar excepciones detectadas y continuar el procesamiento utilizando únicamente los datos válidos. De esta manera, el sistema puede operar sobre archivos parcialmente corruptos sin detener la ejecución total del análisis.

Adicionalmente, el procesamiento se ejecuta utilizando diferentes configuraciones de paralelismo mediante múltiples hilos de CPU, con el propósito de analizar el impacto del manejo de errores sobre el rendimiento del sistema. Durante las pruebas experimentales se registran métricas como:

- Tiempo total de procesamiento.
- Cantidad de registros válidos.
- Cantidad de registros inválidos.
- Número de excepciones capturadas.
- Speedup.
- Eficiencia paralela.

Esta implementación permite evaluar no solo el rendimiento del procesamiento paralelo sobre archivos menos optimizados, sino también la robustez del sistema frente a escenarios reales de calidad de datos, donde la validación y el manejo de excepciones representan un componente fundamental del procesamiento masivo de información.

6.1. Manejo de errores y excepciones

Uno de los objetivos principales del presente proyecto consiste en evaluar el comportamiento del procesamiento paralelo frente a escenarios reales de calidad de datos. En muchos entornos de ingeniería de datos, los archivos procesados pueden contener registros incompletos, inconsistencias de formato o valores inválidos que afectan la ejecución de las operaciones analíticas.

Con el propósito de simular este tipo de situaciones, se desarrolló un proceso de generación de archivos CSV con errores controlados a partir del dataset original almacenado en formato Apache Parquet. Estos errores fueron incorporados de manera aleatoria sobre diferentes columnas del dataset, permitiendo construir un entorno experimental orientado a evaluar la robustez del sistema frente a datos inconsistentes.

Entre los principales tipos de errores utilizados en el experimento se encuentran:

- Valores nulos o vacíos.
- Datos fuera de rango.
- Valores de texto en columnas numéricas.
- Fechas inválidas o mal formateadas.
- Registros incompletos.
- Valores atípicos extremos.

Durante el procesamiento de los archivos CSV, el sistema implementa rutinas de validación de datos utilizando estructuras try/except en Python. Estas rutinas permiten detectar registros inválidos sin detener completamente la ejecución del procesamiento analítico.

Cuando un registro presenta inconsistencias, el sistema captura la excepción correspondiente, registra el error detectado y continúa procesando el resto de los datos válidos. Este enfoque permite mantener la continuidad del procesamiento incluso en presencia de archivos parcialmente corruptos. El manejo de excepciones implementado incluye:

- Validación de tipos de datos.
- Conversión segura de valores numéricos.
- Verificación de fechas válidas.
- Captura de errores de lectura y conversión.
- Registro de excepciones detectadas.
- Conteo de registros válidos e inválidos.

Aunque estas operaciones introducen un costo computacional adicional, representan un componente fundamental en escenarios reales de procesamiento masivo de datos, donde la calidad de la información no siempre puede garantizarse completamente.

A través de esta estrategia, el proyecto busca analizar el impacto que tienen la validación de datos y el manejo de excepciones sobre el rendimiento del procesamiento paralelo, comparando un entorno optimizado basado en Apache Parquet frente a un entorno de datos menos estructurados basado en archivos CSV con errores controlados.

6.2. Configuración del paralelismo:

El procesamiento experimental desarrollado en este proyecto se ejecutó sobre un entorno multinúcleo utilizando diferentes configuraciones de paralelismo, con el propósito de evaluar el impacto del número de hilos de CPU sobre el rendimiento del procesamiento analítico. Las pruebas fueron realizadas sobre un equipo ASUS ROG Strix equipado con un procesador AMD Ryzen 9 5900HX, el cual cuenta con 8 núcleos físicos y 16 hilos lógicos de ejecución. Esta arquitectura permitió ejecutar operaciones analíticas paralelas utilizando diferentes niveles de concurrencia.

Durante el experimento se configuraron distintos escenarios de paralelismo utilizando la siguiente cantidad de hilos:

Tabla 3 – Configuración de los hilos

Configuración	Número de hilos utilizados
Configuración 1	1 hilo
Configuración 2	2 hilos
Configuración 3	4 hilos
Configuración 4	8 hilos
Configuración 5	16 hilos

En el caso del procesamiento mediante DuckDB, el número de hilos fue controlado utilizando la instrucción:

SET threads = N

donde N representa la cantidad de hilos asignados para la ejecución de cada consulta analítica.

Para el procesamiento de archivos CSV mediante Python, se utilizaron mecanismos de procesamiento concurrente orientados a ejecutar operaciones analíticas y validaciones de datos utilizando múltiples hilos de ejecución.

Cada escenario experimental fue ejecutado múltiples veces con el fin de reducir variaciones ocasionadas por procesos del sistema operativo, caché de memoria o fluctuaciones temporales de carga del procesador. Adicionalmente, se realizaron ejecuciones de calentamiento (warmup) previas a la medición oficial de los tiempos, con el objetivo de estabilizar las métricas obtenidas.

La configuración del paralelismo permitió evaluar métricas de escalabilidad mediante indicadores como:

- Tiempo promedio de ejecución.
- Speedup.
- Eficiencia paralela.

El speedup se calculó comparando el tiempo de ejecución obtenido utilizando múltiples hilos frente al tiempo registrado en la ejecución secuencial con un único hilo. Por su parte, la eficiencia paralela permitió analizar el grado de aprovechamiento de los recursos de procesamiento disponibles a medida que aumentaba el nivel de concurrencia.

Esta estrategia experimental permitió observar el comportamiento del sistema frente al incremento progresivo del paralelismo, identificando tanto mejoras en el rendimiento como pérdidas de eficiencia ocasionadas por overhead de sincronización y administración de recursos.

7. Operaciones analíticas de prueba:

Con el propósito de evaluar el comportamiento del procesamiento paralelo sobre grandes volúmenes de datos, se definió un conjunto de operaciones analíticas representativas de escenarios comunes en sistemas de análisis de datos y entornos OLAP. Las consultas seleccionadas permiten analizar diferentes niveles de complejidad computacional, incluyendo operaciones simples de agregación, cálculos estadísticos, filtrado de registros y agrupamientos sobre millones de datos.

Las operaciones analíticas fueron ejecutadas tanto sobre archivos Apache Parquet utilizando DuckDB como sobre archivos CSV con errores controlados utilizando rutinas de procesamiento y validación desarrolladas en Python. Las consultas implementadas en el experimento fueron las siguientes:

7.1. Conteo total de registros (COUNT)

Esta operación permite medir el rendimiento básico de lectura y conteo sobre grandes volúmenes de datos.

Ejemplo:

```
SELECT COUNT(*)  
FROM dataset
```

7.2. Suma total de pasajeros (SUM)

Permite evaluar operaciones de agregación numérica sobre columnas completas del dataset.

Ejemplo:

```
SELECT SUM(passenger_count)
FROM dataset
```

7.3. Promedio de distancia recorrida (AVG)

Evalúa cálculos estadísticos sobre millones de registros numéricos.

Ejemplo:

```
SELECT AVG(trip_distance)
FROM dataset
```

7.4. Agrupación por cantidad de pasajeros (GROUP BY)

Esta consulta introduce operaciones de agrupamiento y agregación simultánea, representando escenarios analíticos de mayor complejidad.

Ejemplo:

```
SELECT passenger_count,
COUNT(*),
AVG(trip_distance),
SUM(fare_amount)
FROM dataset
GROUP BY passenger_count
```

7.5. Filtrado y agrupamiento analítico (WHERE + GROUP BY)

Esta operación combina filtrado de registros con agrupamientos y agregaciones, permitiendo evaluar consultas analíticas complejas bajo procesamiento paralelo.

Ejemplo:

```
SELECT passenger_count,
COUNT(*),
AVG(trip_distance),
SUM(fare_amount)
FROM dataset
WHERE trip_distance > 5
GROUP BY passenger_count
```

Estas operaciones fueron seleccionadas debido a que representan patrones frecuentes en sistemas de análisis de datos masivos, tales como generación de

estadísticas, agregaciones analíticas, segmentación de información y consultas de inteligencia de negocio.

Adicionalmente, las consultas permiten evaluar distintos niveles de aprovechamiento del paralelismo, ya que operaciones simples como COUNT presentan menores costos computacionales, mientras que consultas con filtros y agrupamientos requieren mayor coordinación entre hilos y procesamiento de estructuras compartidas.

La ejecución de estas operaciones permitió registrar métricas de tiempo de ejecución, speedup y eficiencia paralela bajo diferentes configuraciones de hilos de CPU, facilitando el análisis del comportamiento escalable del sistema frente a distintos tipos de carga analítica.

7.6. Consulta 1 - Conteo total de registros:

La primera operación analítica implementada en el experimento corresponde al conteo total de registros presentes en el dataset. Esta consulta representa una de las operaciones más básicas dentro del análisis de datos, permitiendo medir el comportamiento general de lectura y procesamiento sobre grandes volúmenes de información.

La consulta utilizada fue la siguiente:

```
SELECT COUNT(*)  
FROM dataset
```

El objetivo principal de esta operación consiste en evaluar el rendimiento base del sistema durante la lectura masiva de datos, minimizando la complejidad computacional asociada a cálculos estadísticos o agrupamientos. Debido a su simplicidad, esta consulta permite analizar principalmente el impacto del acceso a disco, lectura de datos y distribución del trabajo entre múltiples hilos de procesamiento.

En el escenario basado en Apache Parquet, la consulta fue ejecutada utilizando DuckDB bajo diferentes configuraciones de paralelismo, permitiendo medir el comportamiento escalable del motor analítico frente a operaciones de conteo sobre millones de registros. Por otro lado, en el escenario basado en archivos CSV con errores controlados, el conteo de registros se complementó con rutinas de validación y manejo de excepciones, verificando la integridad de cada registro antes de incluirlo en el procesamiento analítico.

Aunque esta operación presenta una baja complejidad computacional en comparación con consultas que incluyen agrupamientos o filtrados, resulta útil como línea base experimental para comparar el comportamiento del sistema bajo distintos niveles de paralelismo. Durante las pruebas experimentales se registraron métricas como:

- Tiempo de ejecución.
- Speedup.
- Eficiencia paralela.
- Cantidad de registros válidos e inválidos.
- Número de excepciones detectadas en archivos CSV.

Los resultados obtenidos evidenciaron que esta consulta presenta mejoras moderadas al incrementar el número de hilos de procesamiento, debido a que las operaciones de conteo poseen una menor carga computacional y un menor nivel de paralelización efectiva en comparación con consultas analíticas más complejas.

7.7. Consulta 2 - Filtrado de registros:

La segunda operación analítica implementada en el experimento corresponde al filtrado de registros utilizando condiciones sobre columnas numéricas del dataset. Este tipo de consultas es ampliamente utilizado en entornos analíticos y sistemas de inteligencia de negocio, ya que permite seleccionar subconjuntos específicos de información a partir de criterios definidos. La consulta utilizada en el experimento fue la siguiente:

```
SELECT *
FROM dataset
WHERE trip_distance > 5
```

El objetivo principal de esta operación consiste en evaluar el comportamiento del procesamiento paralelo durante la aplicación de filtros sobre grandes volúmenes de datos. En este caso, únicamente se seleccionan los registros correspondientes a viajes cuya distancia recorrida sea superior a cinco unidades.

En el escenario basado en Apache Parquet, esta consulta permite aprovechar una de las principales ventajas del almacenamiento columnar: la lectura selectiva de datos. DuckDB puede procesar únicamente las columnas necesarias para aplicar el filtro, reduciendo el volumen de información leído desde disco y mejorando el rendimiento general de la consulta. Adicionalmente, el filtrado de registros favorece el procesamiento paralelo debido a que el dataset puede dividirse entre múltiples hilos de ejecución, permitiendo aplicar la condición de filtrado simultáneamente sobre diferentes fragmentos de datos.

En el escenario basado en archivos CSV con errores controlados, el procesamiento incluyó validaciones adicionales sobre los valores utilizados en la condición del filtro. Esto permitió detectar registros inválidos, valores fuera de rango o inconsistencias de formato antes de ejecutar las operaciones analíticas correspondientes.

Durante la ejecución de esta consulta se registraron métricas relacionadas con:

- Tiempo de ejecución.
- Speedup.
- Eficiencia paralela.
- Cantidad de registros filtrados.
- Número de registros inválidos detectados.
- Excepciones generadas durante el procesamiento.

Los resultados experimentales mostraron que las operaciones de filtrado presentan un mejor aprovechamiento del paralelismo en comparación con consultas simples de conteo, debido a que la reducción del volumen de datos procesados permite disminuir la carga computacional de etapas posteriores del análisis.

7.8. Consulta 3 - Agregación por categoría:

La tercera operación analítica implementada en el experimento corresponde a una consulta de agregación y agrupamiento de datos utilizando la cláusula GROUP BY. Este tipo de operaciones es ampliamente utilizado en sistemas analíticos y herramientas de inteligencia de negocio para resumir información y generar estadísticas sobre grandes volúmenes de datos.

La consulta utilizada fue la siguiente:

```
SELECT passenger_count,
      COUNT(*) AS total_viajes,
      AVG(trip_distance) AS distancia_promedio,
      SUM(fare_amount) AS total_recaudado
FROM dataset
GROUP BY passenger_count
```

El objetivo principal de esta operación consiste en analizar el comportamiento del procesamiento paralelo durante la ejecución de agrupamientos y agregaciones simultáneas sobre millones de registros.

En este caso, los datos son agrupados según la cantidad de pasajeros registrada en cada viaje, calculando para cada categoría el número total de viajes, la distancia promedio recorrida y el valor total recaudado.

A diferencia de consultas simples de conteo o filtrado, las operaciones de agrupamiento presentan una mayor complejidad computacional debido a que requieren:

- División de datos entre múltiples hilos.
- Construcción de estructuras temporales de agregación.
- Sincronización parcial entre procesos de agrupamiento.
- Consolidación final de resultados.

Estas características convierten a las operaciones GROUP BY en una carga analítica representativa para evaluar el comportamiento del paralelismo en sistemas de procesamiento masivo de datos.

En el escenario basado en Apache Parquet, DuckDB aprovecha el almacenamiento columnar y el procesamiento paralelo interno para ejecutar las agregaciones de manera eficiente sobre múltiples fragmentos del dataset.

Por otro lado, en el escenario basado en archivos CSV con errores controlados, el procesamiento incorpora validaciones adicionales sobre las columnas utilizadas en las agregaciones, verificando que los valores numéricos puedan convertirse correctamente antes de participar en los cálculos estadísticos.

Durante las pruebas experimentales se registraron métricas como:

- Tiempo de ejecución.
- Speedup.
- Eficiencia paralela.
- Cantidad de grupos generados.
- Registros válidos e inválidos.
- Número de excepciones capturadas.

Los resultados obtenidos evidenciaron que las consultas de agregación presentan un mayor costo computacional en comparación con operaciones simples de lectura o conteo. Sin embargo, también mostraron un mejor aprovechamiento del paralelismo debido a que las tareas de agrupamiento y agregación pueden distribuirse parcialmente entre múltiples hilos de procesamiento.

7.9. Consulta 4 - Ordenamiento de registros:

Otra de las operaciones analíticas consideradas dentro del experimento corresponde al ordenamiento de registros mediante la cláusula ORDER BY. Este tipo de consultas es ampliamente utilizado en sistemas analíticos y plataformas de inteligencia de negocio para organizar resultados según criterios específicos de clasificación. Una consulta representativa de este tipo puede expresarse de la siguiente forma:

```
SELECT passenger_count,  
       fare_amount,  
       trip_distance  
FROM dataset  
ORDER BY fare_amount DESC
```

El objetivo principal de esta operación consiste en analizar el comportamiento del procesamiento paralelo durante tareas de clasificación de grandes volúmenes de información.

Las operaciones de ordenamiento presentan una complejidad computacional considerable debido a que requieren reorganizar grandes cantidades de datos en memoria o en almacenamiento temporal. En escenarios de procesamiento paralelo, este tipo de consultas implica dividir el dataset entre múltiples hilos de ejecución, ordenar parcialmente cada fragmento y posteriormente combinar los resultados en una etapa final de consolidación.

Debido a estas características, las consultas ORDER BY representan una carga analítica importante para evaluar:

- Uso de memoria.
- Coordinación entre hilos.
- Overhead de sincronización.
- Escalabilidad del procesamiento paralelo.

En el caso de Apache Parquet y DuckDB, el procesamiento columnar y las optimizaciones internas del motor analítico permiten ejecutar operaciones de ordenamiento de manera más eficiente sobre grandes volúmenes de datos.

Por otro lado, en el escenario basado en archivos CSV con errores controlados, el ordenamiento requiere validaciones adicionales sobre las columnas utilizadas como criterio de clasificación, verificando la integridad y consistencia de los valores antes de realizar el proceso de ordenamiento.

Aunque las operaciones de ordenamiento no constituyeron la carga principal del benchmark experimental, su inclusión dentro del estudio permite identificar uno de los escenarios clásicos de alto costo computacional en sistemas de análisis de datos masivos y procesamiento paralelo.

7.10. Consulta 5 - Agregación temporal:

La quinta operación analítica considerada dentro del experimento corresponde a consultas de agregación temporal utilizando columnas de fecha y hora presentes en el dataset NYC Yellow Taxi Trip Records. Este tipo de operaciones es ampliamente utilizado en sistemas de análisis de datos, inteligencia de negocio y plataformas de monitoreo, debido a que permite identificar patrones de comportamiento a lo largo del tiempo.

Una consulta representativa de este tipo puede expresarse de la siguiente forma:

```
SELECT DATE(tpep_pickup_datetime) AS fecha,  
        COUNT(*) AS total_viajes,  
        AVG(trip_distance) AS distancia_promedio  
FROM dataset  
GROUP BY fecha  
ORDER BY fecha
```

El objetivo principal de esta operación consiste en evaluar el comportamiento del procesamiento paralelo durante tareas de agrupamiento temporal sobre grandes volúmenes de información.

Las consultas temporales presentan una complejidad adicional debido a que requieren:

- Conversión y validación de fechas.
- Extracción de componentes temporales.
- Agrupamiento cronológico de registros.
- Ordenamiento de resultados temporales.

En el escenario basado en Apache Parquet, DuckDB permite ejecutar este tipo de operaciones de manera eficiente gracias a su procesamiento columnar y al soporte optimizado para funciones temporales sobre grandes datasets.

Por otro lado, en el escenario basado en archivos CSV con errores controlados, las operaciones temporales requieren validaciones adicionales sobre los campos de fecha y hora, debido a que los registros pueden contener:

- Fechas inválidas.
- Formatos inconsistentes.
- Valores nulos.
- Datos incompletos.

Durante el procesamiento, el sistema implementa mecanismos de manejo de excepciones para evitar que errores en los registros temporales interrumpan completamente la ejecución de las consultas analíticas. Las operaciones de agregación temporal permiten evaluar escenarios analíticos más cercanos a aplicaciones reales de procesamiento de datos, donde el análisis cronológico constituye uno de los componentes más utilizados en sistemas de monitoreo, movilidad urbana y generación de estadísticas operacionales.

Adicionalmente, este tipo de consultas permite analizar el impacto del paralelismo sobre operaciones que combinan filtrado, agrupamiento y procesamiento de funciones temporales simultáneamente.

8. Métricas de evaluación

Con el propósito de analizar el comportamiento del procesamiento paralelo sobre grandes volúmenes de datos, el presente proyecto definió un conjunto de métricas orientadas a evaluar tanto el rendimiento del sistema como su robustez frente a escenarios con errores e inconsistencias en los datos.

Las métricas seleccionadas permiten comparar el comportamiento del procesamiento analítico bajo diferentes configuraciones de paralelismo y distintos

formatos de almacenamiento. Las principales métricas evaluadas durante el experimento fueron las siguientes:

8.1. Tiempo de ejecución

El tiempo de ejecución representa la duración total requerida para completar una operación analítica determinada. Esta métrica permite medir el rendimiento general del sistema bajo diferentes configuraciones de hilos de CPU.

Los tiempos fueron registrados utilizando temporizadores de alta precisión en Python mediante la función `time.perf_counter()`, permitiendo obtener mediciones detalladas en segundos para cada consulta ejecutada. Las pruebas fueron repetidas múltiples veces con el fin de reducir variaciones ocasionadas por procesos del sistema operativo, efectos de caché y fluctuaciones temporales de carga del procesador.

8.2. Speedup

El speedup permite medir la mejora en el tiempo de ejecución cuando se utilizan múltiples núcleos de procesamiento. Se calcula mediante la siguiente expresión:

$$Speedup = \frac{T_1}{T_N}$$

donde:

- T_1 corresponde al tiempo de ejecución utilizando un solo hilo
- T_N corresponde al tiempo de ejecución utilizando N hilos

Esta métrica permite evaluar qué tan efectiva es la paralelización del proceso, donde el tiempo secuencial corresponde al tiempo de ejecución utilizando un único hilo de procesamiento y el tiempo paralelo representa el tiempo obtenido utilizando múltiples hilos. El análisis del speedup permitió evaluar el nivel de escalabilidad alcanzado por las operaciones analíticas implementadas en el experimento.

8.3. Eficiencia paralela

La eficiencia paralela permite medir el grado de aprovechamiento de los recursos de procesamiento disponibles. Se calcula mediante la siguiente expresión:

$$Eficiencia = \frac{Speedup}{N}$$

donde:

- *Speedup* es la aceleración obtenida
- N es el número de hilos utilizados

La eficiencia paralela permite identificar pérdidas de rendimiento ocasionadas por overhead de sincronización, competencia por memoria y coordinación entre hilos de ejecución. Una eficiencia cercana a 1 indica que el paralelismo está siendo aprovechado de forma óptima. Durante el experimento se registrarán los resultados en una tabla similar a la siguiente:

Tabla 4 – Modelo de tabla para registrar resultados

Hilos	Tiempo consulta (s)	Speedup	Eficiencia
1	120	1.0	1.0
2	70	1.71	0.85
4	40	3.0	0.75
8	25	4.8	0.60
16	15	8.0	0.50

Esta tabla permitirá analizar cómo varía el rendimiento del sistema al aumentar el nivel de paralelismo.

Dado que el sistema implementado en Python incorporará mecanismos de manejo de excepciones, también se registrará la cantidad de errores detectados durante el procesamiento de los datos. Entre los errores que pueden registrarse se incluyen:

- Registros vacíos
- Valores nulos
- Errores de formato
- Conversiones inválidas de tipo de datos

Este registro permitirá evaluar la robustez del sistema frente a datasets con posibles inconsistencias.

8.4.Registros válidos e inválidos

En el escenario experimental basado en archivos CSV con errores controlados, se registró la cantidad de registros válidos e inválidos detectados durante el procesamiento. Esta métrica permitió evaluar el impacto de la calidad de los datos sobre el rendimiento del sistema y sobre los mecanismos de validación implementados en Python.

8.5.Excepciones capturadas

Durante el procesamiento de archivos CSV se contabilizó el número de excepciones detectadas y manejadas por el sistema. Entre las excepciones consideradas se encuentran:

- Errores de conversión numérica.
- Fechas inválidas.

- Valores nulos.
- Registros incompletos.
- Inconsistencias de formato.

El registro de excepciones permitió evaluar la robustez del sistema frente a escenarios reales de procesamiento de datos corruptos o inconsistentes.

8.6. Escalabilidad del procesamiento

La combinación de métricas como tiempo de ejecución, speedup y eficiencia paralela permitió analizar el comportamiento escalable del sistema frente al incremento progresivo del número de hilos de CPU.

A través de estas métricas fue posible identificar consultas con mayor aprovechamiento del paralelismo, así como escenarios donde el overhead de sincronización y manejo de errores impactó negativamente el rendimiento del procesamiento.

9. Procedimiento experimental

El procedimiento experimental desarrollado en el presente proyecto se estructuró en diferentes etapas orientadas a evaluar el comportamiento del procesamiento paralelo sobre grandes volúmenes de datos bajo dos escenarios experimentales: un entorno optimizado utilizando archivos Apache Parquet y un entorno basado en archivos CSV con errores controlados.

El experimento fue ejecutado utilizando el dataset NYC Yellow Taxi Trip Records y diferentes configuraciones de paralelismo sobre un entorno multinúcleo. El procedimiento experimental se desarrolló de la siguiente manera:

9.1. Preparación del dataset

Inicialmente se seleccionó el dataset NYC Yellow Taxi Trip Records correspondiente al periodo 2018–2023. Los archivos originales fueron utilizados en formato Apache Parquet debido a sus características de almacenamiento columnar y optimización para procesamiento analítico. Los datos utilizados incluyen información relacionada con:

- Distancia de los viajes.
- Número de pasajeros.
- Tipo de pago.
- Fechas y horas de viaje.
- Tarifas y valores monetarios.

9.2. Configuración del entorno experimental

El entorno de pruebas fue configurado sobre un equipo con arquitectura multinúcleo, utilizando un procesador AMD Ryzen 9 5900HX con 8 núcleos físicos y 16 hilos lógicos. Las pruebas experimentales fueron ejecutadas utilizando diferentes configuraciones de paralelismo:

- 1 hilo
- 2 hilos
- 4 hilos
- 8 hilos
- 16 hilos

9.3. Ejecución del escenario Apache Parquet

En el primer escenario experimental se ejecutaron consultas analíticas sobre archivos Apache Parquet utilizando DuckDB como motor analítico. Las consultas fueron ejecutadas utilizando diferentes niveles de paralelismo mediante la instrucción:

SET threads = N

Durante esta etapa se realizaron operaciones analíticas de:

- Conteo de registros.
- Filtrado de información.
- Agregaciones numéricas.
- Agrupamientos por categoría.
- Consultas temporales.

Cada consulta fue ejecutada múltiples veces para reducir variaciones en las mediciones y mejorar la estabilidad de los resultados experimentales.

9.4. Generación de archivos CSV con errores controlados

A partir de los archivos Apache Parquet originales se desarrolló un proceso de conversión hacia formato CSV incorporando errores aleatorios sobre diferentes columnas del dataset. Entre los errores generados se encuentran:

- Valores nulos.
- Datos fuera de rango.
- Registros incompletos.
- Fechas inválidas.
- Valores de texto en columnas numéricas.

El objetivo de esta etapa fue construir un escenario experimental orientado a evaluar la robustez del procesamiento frente a datos inconsistentes.

9.5. Procesamiento de archivos CSV

Los archivos CSV generados fueron procesados utilizando rutinas desarrolladas en Python, incorporando mecanismos de validación de datos y manejo de excepciones mediante estructuras *try/except*. Durante esta etapa se verificó:

- Integridad de registros.
- Conversión correcta de valores numéricos.
- Validación de fechas.
- Detección de inconsistencias.

Cuando un registro presentaba errores, el sistema registraba la excepción detectada y continuaba el procesamiento utilizando únicamente los datos válidos.

9.6. Registro de métricas experimentales

Durante la ejecución de ambos escenarios experimentales se registraron métricas relacionadas con:

- Tiempo de ejecución.
- Speedup.
- Eficiencia paralela.
- Registros válidos e inválidos.
- Número de excepciones detectadas.

Las métricas fueron almacenadas en archivos CSV para facilitar el análisis posterior y la generación de gráficas comparativas.

9.7. Análisis comparativo de resultados

Finalmente, los resultados obtenidos fueron analizados comparando el comportamiento del procesamiento paralelo bajo los dos escenarios experimentales propuestos:

- Procesamiento optimizado utilizando Apache Parquet.
- Procesamiento con validación y manejo de errores utilizando archivos CSV.

Este análisis permitió evaluar tanto el rendimiento del procesamiento analítico como el impacto que tienen la validación de datos y el manejo de excepciones sobre la escalabilidad y eficiencia del sistema.

10. Presentación y análisis de resultados

La presente sección expone los resultados obtenidos durante la ejecución de los experimentos desarrollados sobre el dataset NYC Yellow Taxi Trip Records

utilizando diferentes configuraciones de paralelismo y distintos formatos de almacenamiento. Los resultados fueron organizados en dos escenarios experimentales principales:

- Escenario 1: Procesamiento optimizado utilizando Apache Parquet y DuckDB.
- Escenario 2: Procesamiento de archivos CSV con errores controlados y manejo de excepciones.

El análisis realizado considera métricas de rendimiento, escalabilidad y robustez, permitiendo evaluar el comportamiento del sistema frente a distintos tipos de carga analítica y calidad de datos.

10.1. Resultados experimentales utilizando Apache Parquet

En el primer escenario experimental se ejecutaron consultas analíticas sobre archivos Apache Parquet utilizando DuckDB como motor de procesamiento. Las pruebas fueron realizadas utilizando diferentes configuraciones de hilos de CPU:

- 1 hilo
- 2 hilos
- 4 hilos
- 8 hilos
- 16 hilos

Durante las pruebas se registraron métricas relacionadas con:

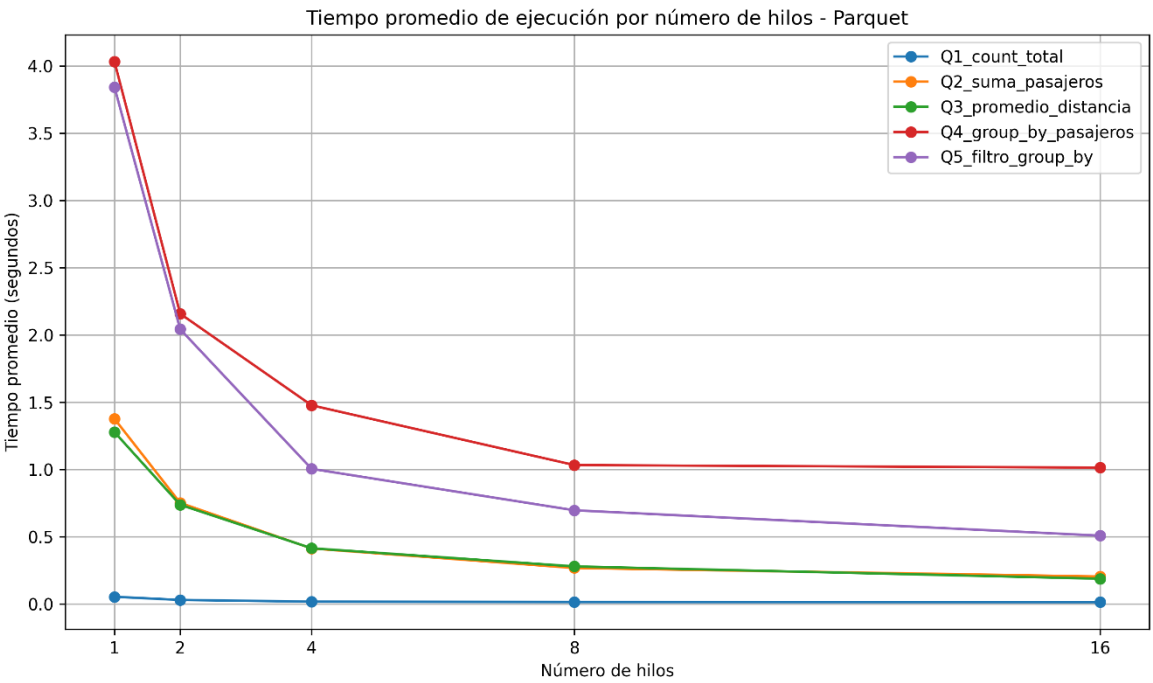
- Tiempo de ejecución.
- Speedup.
- Eficiencia paralela.

10.2. Resultados de tiempo de ejecución

Tabla 5 - Tiempo promedio de ejecución (segundos)

Consulta	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos
 COUNT(*)	0.054	0.029	0.018	0.014	0.013
 SUM(passenger_count)	1.37	0.74	0.40	0.29	0.20
 AVG(trip_distance)	1.36	0.71	0.38	0.28	0.20
 GROUP BY passenger_count	4.03	2.19	1.32	1.02	0.91
 WHERE + GROUP BY	3.84	2.02	1.00	0.70	0.49

Ilustración 1 - Tiempo promedio de ejecución utilizando Apache Parquet.



Análisis: Los resultados evidenciaron una reducción significativa del tiempo de ejecución a medida que aumentó el número de hilos utilizados durante el procesamiento analítico.

Las consultas con operaciones de agregación y filtrado presentaron un mejor aprovechamiento del paralelismo en comparación con consultas simples de conteo. Particularmente, la consulta que combina filtrado y agrupamiento (WHERE + GROUP BY) alcanzó los mejores resultados de rendimiento, reduciendo el tiempo promedio de ejecución desde aproximadamente 3.84 segundos utilizando un único hilo hasta cerca de 0.49 segundos utilizando 16 hilos de procesamiento.

Por otro lado, la consulta COUNT(*) presentó mejoras más moderadas debido a su baja complejidad computacional y al reducido trabajo analítico requerido por hilo.

10.3. Resultados de speedup

Tabla 6 - Speedup máximo obtenido

ID	Consulta	Descripción de la operación	Speedup máximo aproximado	Nivel de escalabilidad
Q1	 COUNT(*)	Conteo simple de registros totales en el dataset.	3.9x 	MODERADO 
Q2	 SUM(passenger_count)	Suma total de pasajeros transportados.	6.7x 	ALTO 
Q3	 AVG(trip_distance)	Promedio de la distancia recorrida en los viajes.	6.8x 	ALTO 
Q4	 GROUP BY passenger_count	Agrupamiento por número de pasajeros y conteo por grupo.	4.4x 	MEDIO-ALTO 
Q5	 WHERE + GROUP BY	Filtrado de registros y agrupamiento por número de pasajeros.	7.7x 	MUY ALTO 

Análisis: Las consultas analíticas complejas mostraron mayores niveles de aceleración paralela. La consulta WHERE + GROUP BY alcanzó el mayor speedup del experimento, aproximándose a 7.7x utilizando 16 hilos de procesamiento.

Estos resultados evidencian que las operaciones que combinan filtrado, agrupamiento y agregación presentan un mejor aprovechamiento del paralelismo multinúcleo en comparación con operaciones simples de conteo.

Las consultas SUM y AVG también mostraron mejoras importantes en el rendimiento, alcanzando speedups superiores a 6x bajo configuraciones de alta concurrencia.

Ilustración 2 - Speedup obtenido utilizando diferentes niveles de paralelismo.

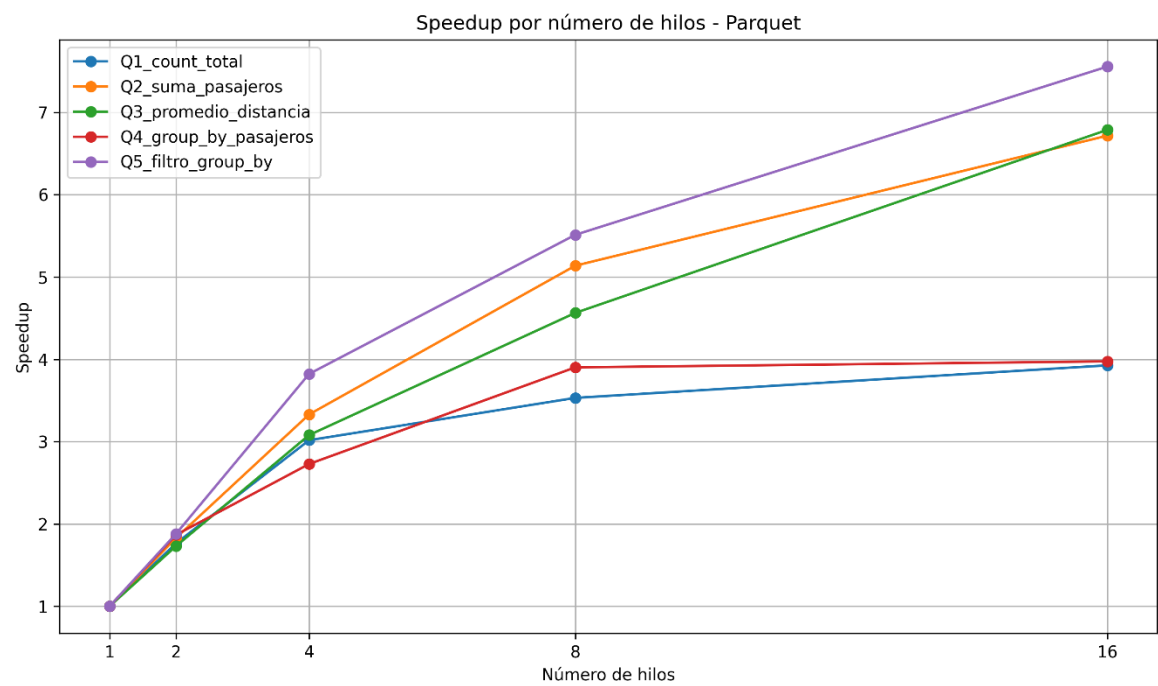


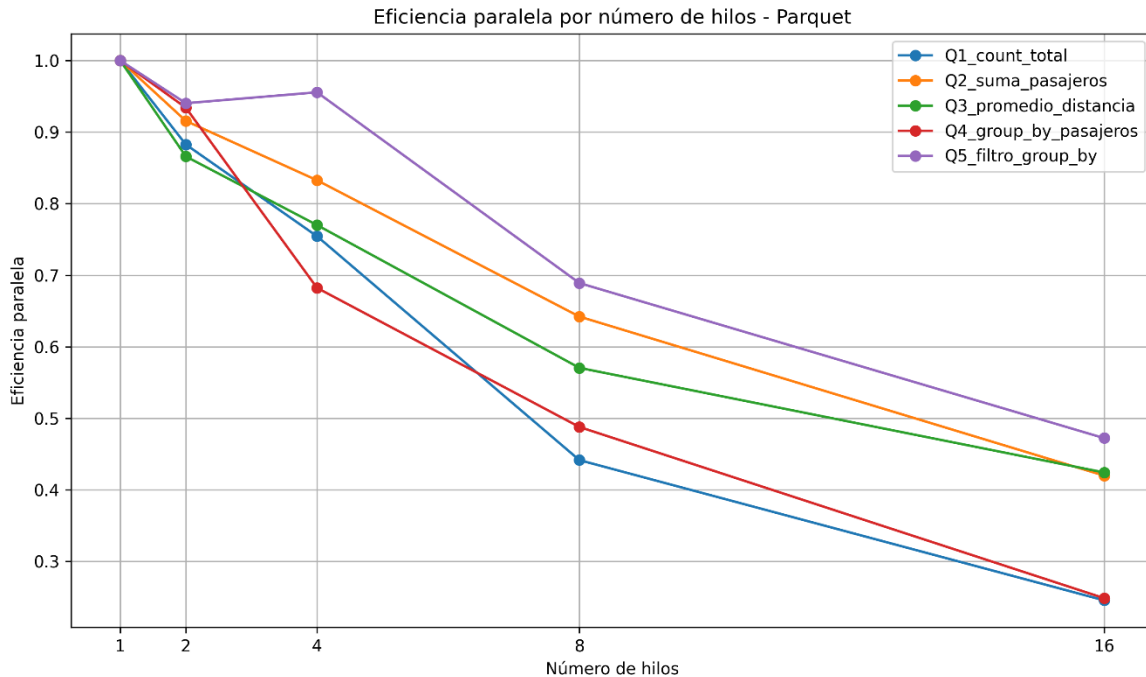
Tabla 7 - Resultados de eficiencia paralela

Consulta	Eficiencia paralela				
	1 hilo	2 hilos	4 hilos	8 hilos	16 hilos
COUNT(*)	1.00	0.88	0.75	0.44	0.25
SUM (passenger_count)	1.00	0.92	0.83	0.64	0.42
AVG (trip_distance)	1.00	0.87	0.77	0.57	0.43
GROUP BY passenger_count	1.00	0.94	0.68	0.49	0.25
WHERE + GROUP BY	1.00	0.94	0.96	0.69	0.47

Interpretación:

- ≥ 0.80 = Excelente
- 0.60 – 0.79 = Buena
- 0.40 – 0.59 = Aceptable
- < 0.40 = Baja

Ilustración 3 - Eficiencia paralela obtenida durante el procesamiento analítico.



Análisis: La eficiencia paralela disminuyó progresivamente al incrementar el número de hilos de ejecución, comportamiento esperado en sistemas de procesamiento paralelo debido al overhead de sincronización, administración de memoria y coordinación entre hilos.

Sin embargo, las consultas complejas mantuvieron mejores niveles de eficiencia en comparación con operaciones simples de conteo, evidenciando un mejor aprovechamiento del paralelismo en cargas analíticas de mayor complejidad.

Particularmente, las consultas que incorporaron filtrado previo de registros lograron mantener niveles de eficiencia superiores durante configuraciones intermedias de paralelismo.

10.4. Análisis del comportamiento de las consultas

El comportamiento experimental de las consultas permitió identificar diferencias importantes en el nivel de aprovechamiento del paralelismo dependiendo del tipo de operación analítica ejecutada.

Las operaciones simples de conteo presentaron menores niveles de aceleración debido a que requieren una carga computacional reducida y generan menor trabajo distribuible entre múltiples hilos.

Por otro lado, las consultas de agregación y agrupamiento mostraron un mejor comportamiento escalable, especialmente aquellas que incorporaron filtros

previos sobre el dataset. Este comportamiento se debe a que la reducción inicial del volumen de datos permite disminuir la carga de procesamiento en etapas posteriores del análisis.

Las operaciones GROUP BY representaron uno de los escenarios de mayor complejidad computacional debido a la necesidad de coordinar estructuras de agregación entre múltiples hilos de ejecución. A pesar de ello, DuckDB logró mantener mejoras significativas de rendimiento gracias a sus mecanismos internos de procesamiento paralelo y optimización sobre archivos Apache Parquet.

Los resultados obtenidos evidencian que el uso de almacenamiento columnar combinado con procesamiento multinúcleo permite reducir considerablemente los tiempos de ejecución sobre grandes volúmenes de datos analíticos.

11.Resultados experimentales utilizando archivos CSV con errores controlados

Con el propósito de evaluar el comportamiento del sistema frente a escenarios de datos no estructurados y con presencia de inconsistencias, se desarrolló una segunda fase experimental utilizando archivos CSV con errores controlados. A diferencia de los archivos Parquet empleados en las pruebas anteriores, los archivos CSV fueron modificados deliberadamente para incorporar registros inválidos, valores atípicos y datos incompletos, permitiendo así analizar el impacto del manejo de excepciones sobre el rendimiento y la escalabilidad del procesamiento paralelo.

Los experimentos fueron ejecutados sobre el mismo entorno de hardware y software descrito previamente, manteniendo las mismas consultas analíticas y configuraciones de paralelismo utilizadas en las pruebas con Parquet. De esta manera, fue posible realizar una comparación directa entre ambos formatos bajo condiciones equivalentes de ejecución.

Los errores introducidos en los archivos CSV incluyeron:

- Valores negativos en campos numéricos,
- Campos vacíos,
- Datos textuales en columnas numéricas,
- Fechas inválidas,
- Registros con valores fuera de rango considerados como outliers.

El procesamiento de estos archivos requirió la implementación de mecanismos adicionales de validación y manejo de excepciones, incrementando la carga computacional del sistema y permitiendo evaluar el costo real asociado a la tolerancia a fallos en entornos de procesamiento masivo de datos. Las pruebas fueron ejecutadas utilizando configuraciones de 1, 2, 4, 8 y 16 hilos/procesos, evaluando las siguientes consultas analíticas:

- Q1: Conteo total de registros.
- Q2: Suma del número de pasajeros.
- Q3: Promedio de distancia recorrida.
- Q4: Agrupación por cantidad de pasajeros.
- Q5: Filtrado y agrupación combinada.

A partir de estos experimentos se analizaron:

- Tiempos promedio de ejecución,
- Speedup,
- Eficiencia paralela,
- Costo de validación,
- Detección de errores,
- Comportamiento de escalabilidad frente al incremento de concurrencia.

11.1. Resultados de validación y manejo de errores

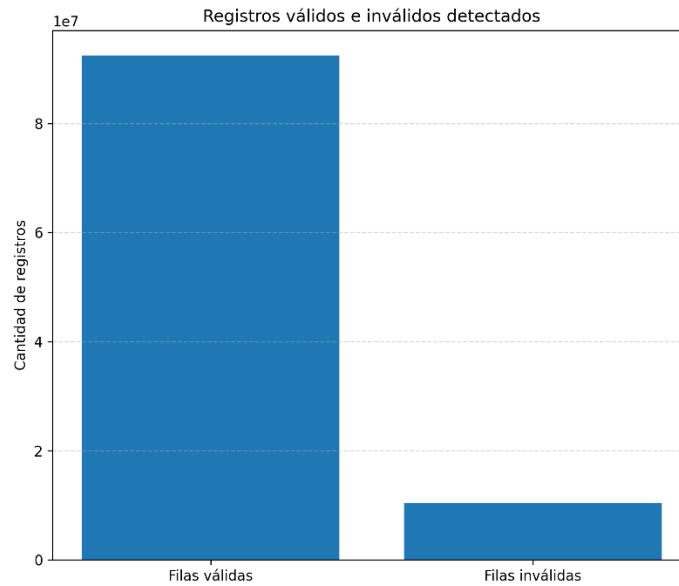
Con el propósito de simular escenarios reales de procesamiento de datos, los archivos CSV utilizados en el experimento fueron modificados mediante la incorporación controlada de diferentes tipos de errores e inconsistencias. Esta estrategia permitió evaluar el comportamiento del sistema frente a datos corruptos o parcialmente inválidos, así como analizar el impacto del manejo de excepciones sobre el rendimiento del procesamiento paralelo.

Durante el procesamiento de los archivos CSV se implementaron mecanismos de validación orientados a verificar la integridad de los registros antes de ejecutar las operaciones analíticas correspondientes. Las rutinas desarrolladas en Python permitieron detectar errores relacionados con:

- Valores nulos,
- Datos fuera de rango,
- Registros incompletos,
- Valores textuales en columnas numéricas,
- Valores atípicos extremos.

Cuando un registro presentaba inconsistencias, el sistema capturaba la excepción correspondiente mediante estructuras try/except, registrando el error detectado y permitiendo continuar el procesamiento del resto de la información válida. Este enfoque evitó que registros defectuosos interrumpieran completamente la ejecución del sistema.

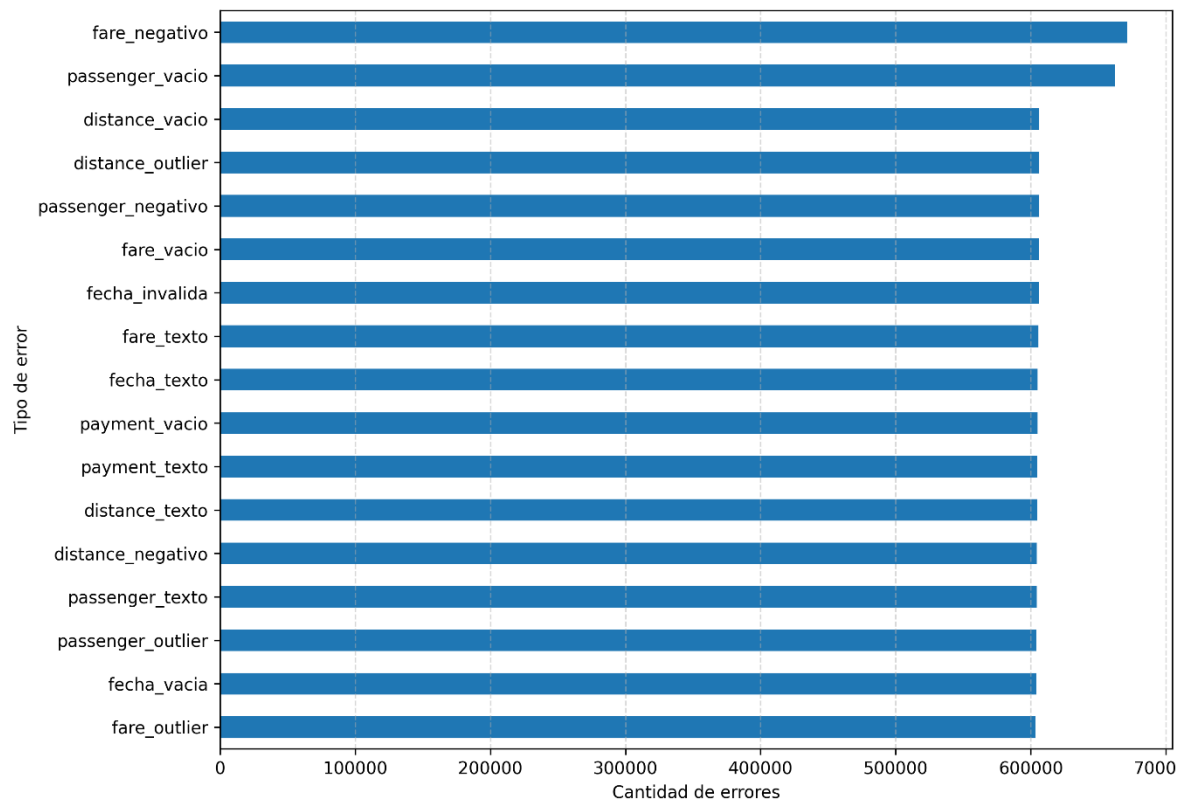
Ilustración 4 – proporción de registros válidos e inválidos



Análisis: Los resultados evidenciaron que el sistema fue capaz de identificar y separar correctamente los registros válidos de aquellos que contenían inconsistencias. Aunque se detectó una cantidad considerable de registros inválidos, el procesamiento logró mantenerse operativo gracias a los mecanismos de validación y manejo de excepciones implementados.

Este comportamiento demuestra que el sistema desarrollado no solamente se enfocó en el rendimiento del procesamiento paralelo, sino también en la robustez y continuidad operativa frente a escenarios reales de calidad de datos. Adicionalmente, se realizó una clasificación de los errores detectados durante el procesamiento de los archivos CSV.

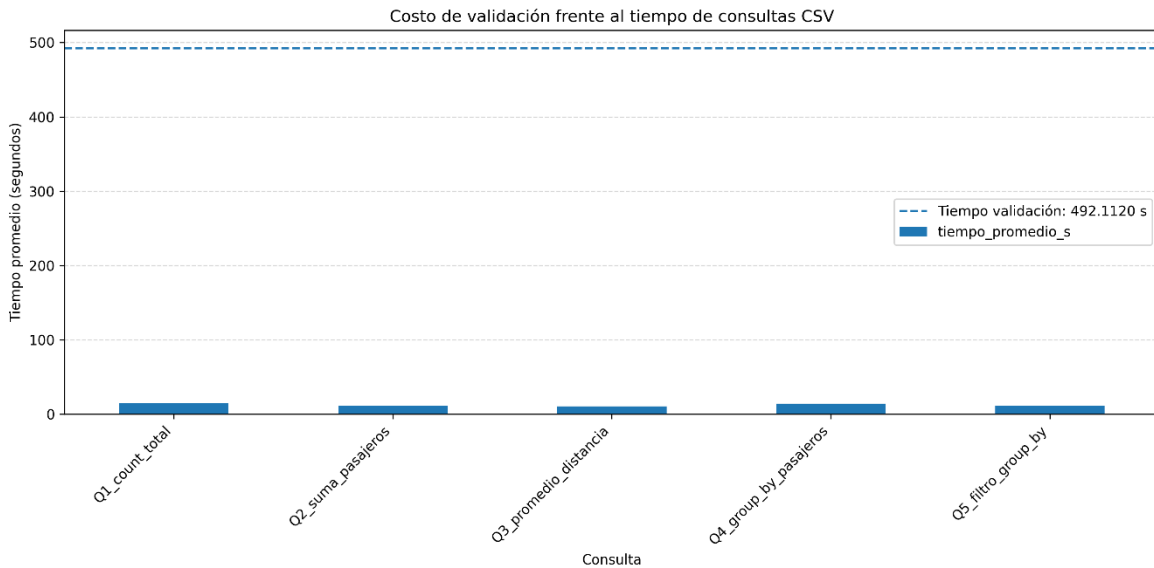
Ilustración 5 – Tipos de errores inyectados en CSV



Análisis: La distribución de errores mostró una presencia relativamente homogénea entre los diferentes tipos de inconsistencias introducidas experimentalmente. Los errores asociados a valores negativos, registros incompletos y datos fuera de rango representaron una parte importante de las excepciones detectadas por el sistema.

La incorporación de múltiples categorías de errores permitió construir un escenario experimental más cercano a condiciones reales de ingeniería de datos, donde la información procesada puede contener inconsistencias provenientes de fallos de captura, problemas de integración o errores de transformación de datos. Finalmente, se evaluó el impacto del proceso de validación sobre el tiempo total de ejecución del sistema.

Ilustración 6 - Overhead generado por validación y manejo de errores



Análisis: Los resultados evidenciaron que las rutinas de validación y manejo de excepciones introdujeron un costo computacional considerable sobre el procesamiento de los archivos CSV. El tiempo consumido por las tareas de validación superó ampliamente el tiempo promedio requerido por las consultas analíticas individuales, demostrando que el manejo de errores representa un factor crítico dentro del rendimiento global del sistema.

Este comportamiento permitió confirmar una de las hipótesis planteadas en el proyecto: la incorporación de mecanismos de tolerancia a fallos y validación de datos incrementa significativamente el overhead computacional del procesamiento paralelo, especialmente en escenarios de alta concurrencia y grandes volúmenes de información.

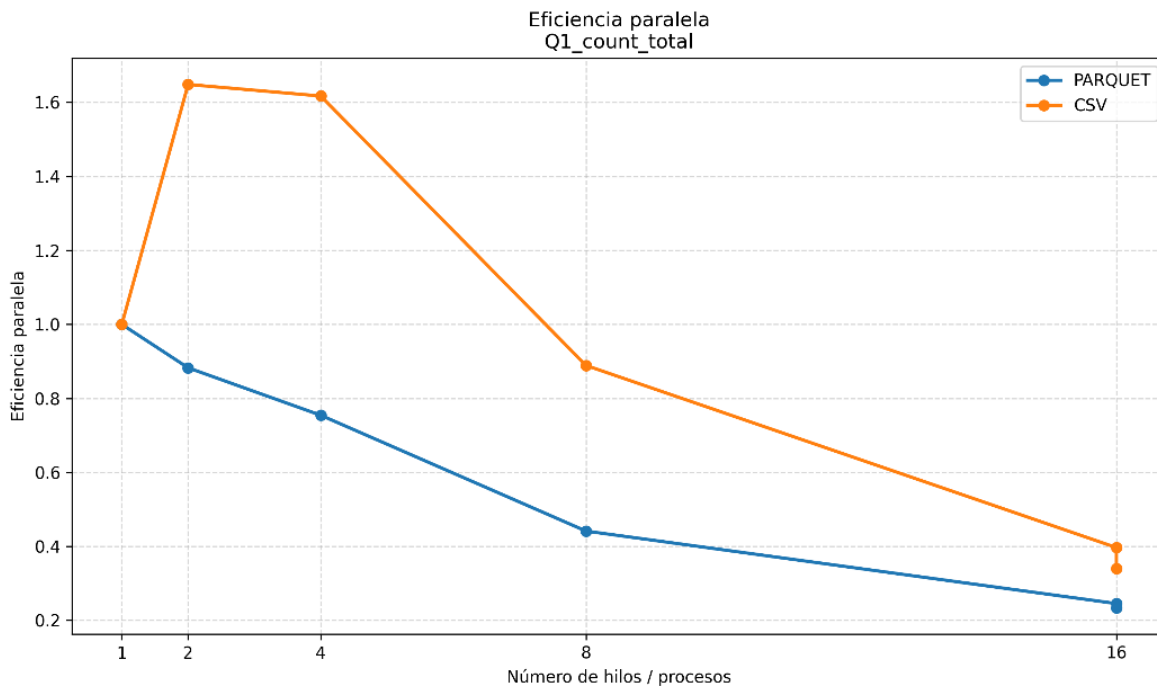
11.2. Resultados de tiempo promedio de ejecución

Con el propósito de evaluar el impacto del paralelismo sobre el procesamiento de archivos CSV con errores controlados, se registraron los tiempos promedio de ejecución para cada una de las consultas analíticas implementadas en el experimento. Las pruebas fueron ejecutadas utilizando configuraciones de 1, 2, 4, 8 y 16 hilos de procesamiento, permitiendo analizar el comportamiento escalable del sistema frente al incremento progresivo de concurrencia.

Las consultas ejecutadas incluyeron operaciones de conteo, agregación, filtrado y agrupamiento sobre grandes volúmenes de datos, incorporando simultáneamente procesos de validación y manejo de excepciones durante la lectura de los archivos CSV.

11.2.1. Consulta Q1 – Conteo total de registros

Ilustración 7 - Tiempo promedio de ejecución Q1 COUNT(*)



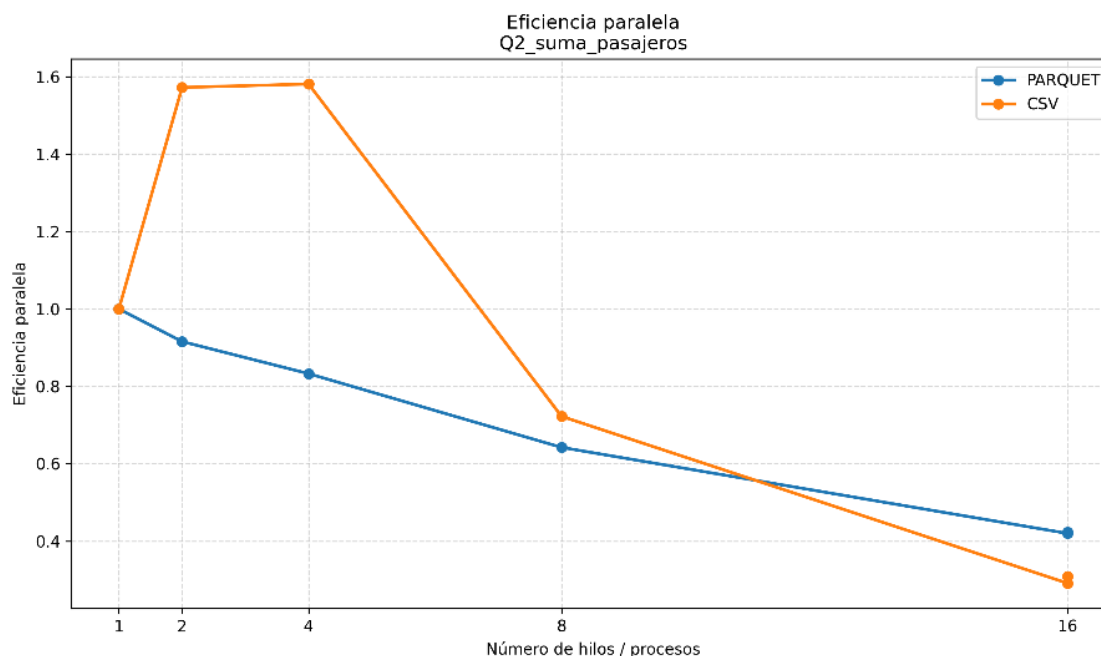
Análisis: La consulta de conteo total de registros presentó una disminución significativa del tiempo de ejecución al incrementar el número de hilos de procesamiento. La reducción más importante se observó entre las configuraciones de 1 y 4 hilos, evidenciando un aprovechamiento inicial efectivo del paralelismo.

Sin embargo, a partir de configuraciones superiores a 8 hilos, la mejora en el rendimiento comenzó a estabilizarse, mostrando un comportamiento típico de saturación ocasionado por overhead de sincronización y competencia por recursos compartidos. Debido a que esta operación presenta una baja complejidad computacional, el beneficio obtenido mediante paralelismo fue menor en comparación con consultas analíticas más complejas.

11.3. Consulta Q2 – Suma total de pasajeros

Análisis: La operación de suma sobre la columna `passenger_count` mostró una mejora progresiva del tiempo de ejecución conforme aumentó el número de hilos disponibles. En comparación con la consulta de conteo, esta operación presentó un mejor aprovechamiento del paralelismo debido al incremento en la carga computacional asociada a la agregación numérica. Los resultados evidenciaron una reducción considerable del tiempo de procesamiento entre las configuraciones de 1 y 8 hilos. No obstante, al utilizar 16 hilos, la ganancia adicional fue más moderada, reflejando nuevamente el impacto del overhead generado por coordinación y validación concurrente de datos.

Ilustración 8 - Tiempo promedio de ejecución Q2 SUM(passenger_count)



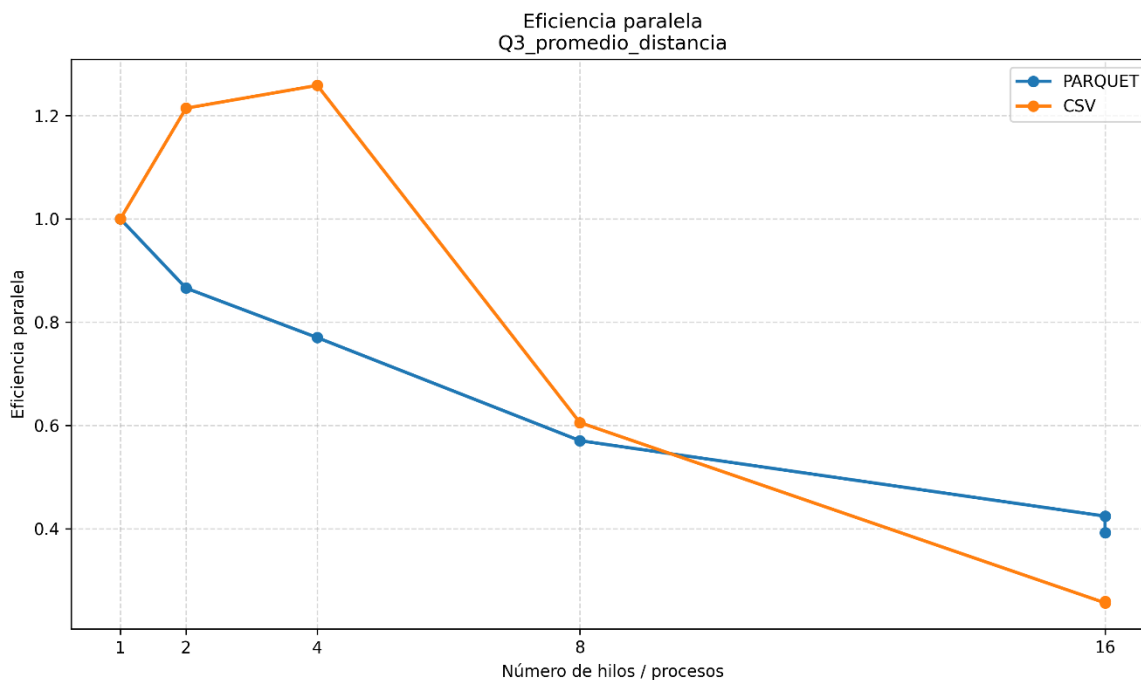
Análisis: La operación de suma sobre la columna `passenger_count` mostró una mejora progresiva del tiempo de ejecución conforme aumentó el número de hilos disponibles. En comparación con la consulta de conteo, esta operación presentó un mejor aprovechamiento del paralelismo debido al incremento en la carga computacional asociada a la agregación numérica. Los resultados evidenciaron una reducción considerable del tiempo de procesamiento entre las configuraciones de 1 y 8 hilos. No obstante, al utilizar 16 hilos, la ganancia adicional fue más moderada, reflejando nuevamente el impacto del overhead generado por coordinación y validación concurrente de datos.

11.4. Consulta Q3 – Promedio de distancia recorrida

Análisis: La consulta orientada al cálculo del promedio de distancia recorrida presentó un comportamiento escalable similar al observado en operaciones de agregación anteriores. El paralelismo permitió reducir significativamente el tiempo de ejecución durante las configuraciones intermedias de concurrencia.

Sin embargo, el procesamiento de esta consulta también requirió validaciones adicionales sobre valores numéricos y detección de registros inválidos, lo que incrementó parcialmente el costo computacional total de la operación. A pesar de ello, los resultados mostraron que el paralelismo logró compensar gran parte del overhead introducido por las rutinas de validación implementadas en Python.

Ilustración 9 - Tiempo promedio de ejecución Q3 AVG(trip_distance)



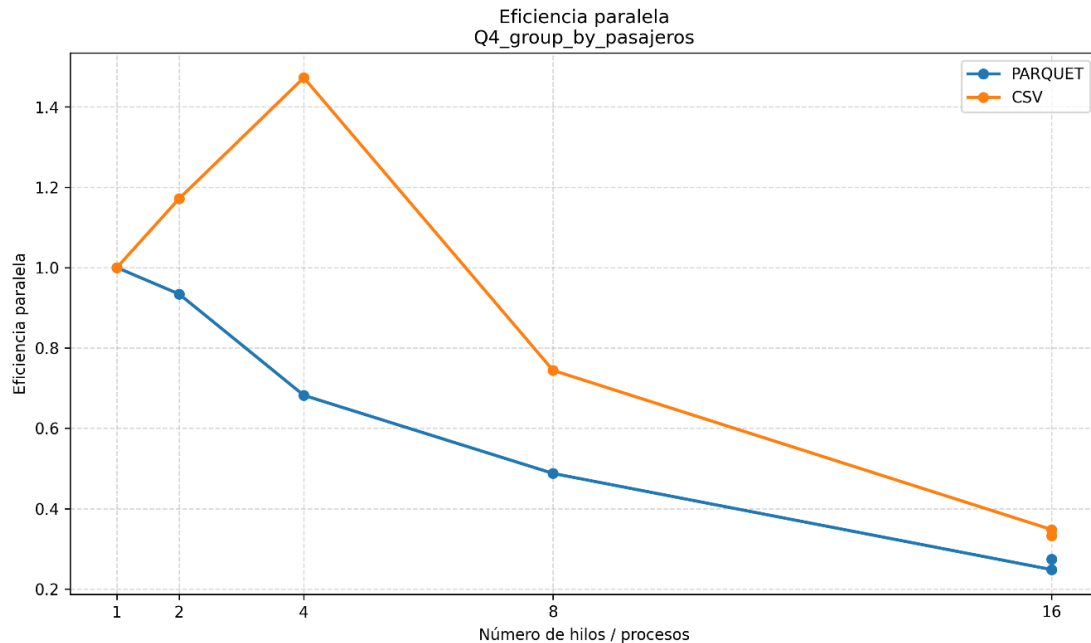
11.4.1. Consulta Q4 – Agrupación por cantidad de pasajeros

Análisis: Las operaciones de agrupamiento presentaron uno de los mayores niveles de complejidad computacional dentro del experimento. La consulta GROUP BY requirió procesos adicionales de agregación, consolidación y coordinación parcial entre múltiples hilos de ejecución.

Aun así, el paralelismo permitió reducir considerablemente los tiempos de ejecución, especialmente entre las configuraciones de 1 y 8 hilos. Los resultados evidenciaron que las consultas analíticas complejas aprovechan de mejor manera el procesamiento multinúcleo debido a la mayor cantidad de trabajo distribuible entre hilos.

No obstante, el incremento hacia 16 hilos mostró señales de saturación y reducción progresiva de eficiencia, asociadas al costo de sincronización y administración concurrente de estructuras de agrupamiento.

Ilustración 10 - Tiempo promedio de ejecución Q4 GROUP BY passenger_count



Análisis: Las operaciones de agrupamiento presentaron uno de los mayores niveles de complejidad computacional dentro del experimento. La consulta GROUP BY requirió procesos adicionales de agregación, consolidación y coordinación parcial entre múltiples hilos de ejecución.

Aun así, el paralelismo permitió reducir considerablemente los tiempos de ejecución, especialmente entre las configuraciones de 1 y 8 hilos. Los resultados evidenciaron que las consultas analíticas complejas aprovechan de mejor manera el procesamiento multinúcleo debido a la mayor cantidad de trabajo distribuible entre hilos.

No obstante, el incremento hacia 16 hilos mostró señales de saturación y reducción progresiva de eficiencia, asociadas al costo de sincronización y administración concurrente de estructuras de agrupamiento.

11.4.2. Consulta Q5 – Filtrado y agrupamiento combinado

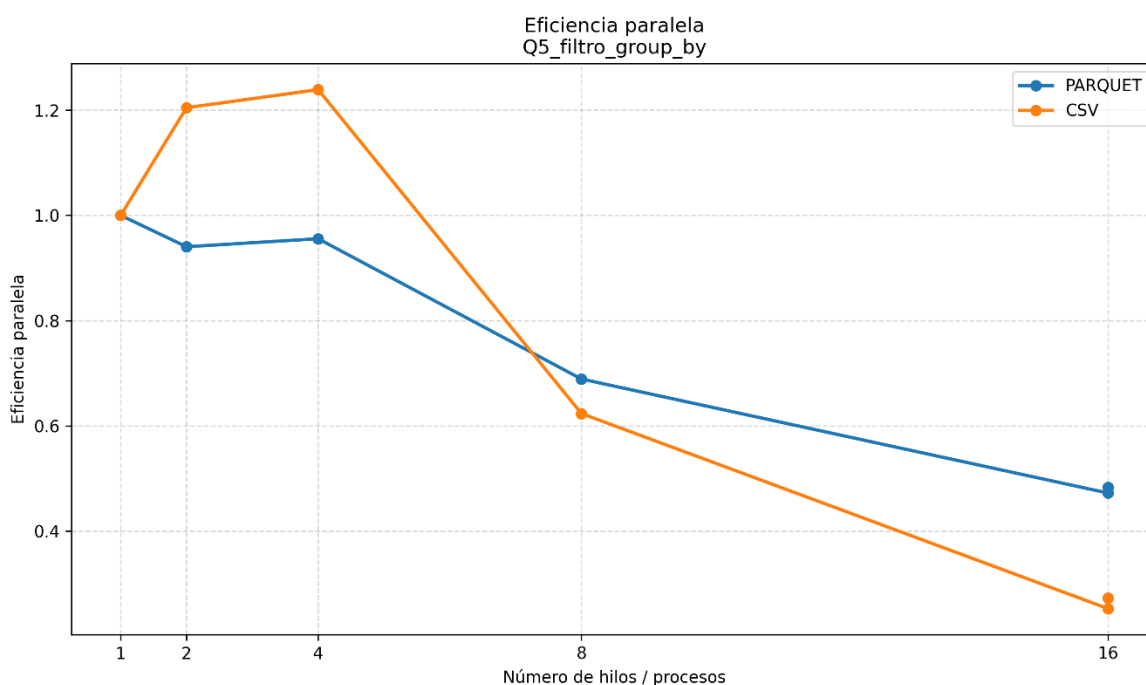
Análisis: La consulta que combina filtrado y agrupamiento presentó el mejor comportamiento escalable dentro del escenario experimental basado en archivos CSV. La aplicación previa del filtro permitió reducir el volumen total de datos procesados durante las etapas posteriores de agrupamiento y agregación.

Este comportamiento favoreció el aprovechamiento del paralelismo, permitiendo obtener reducciones importantes en el tiempo de ejecución al incrementar el número de hilos de procesamiento.

A pesar de que el escenario CSV incorporó validación y manejo de excepciones, la combinación de filtrado y paralelismo permitió amortiguar parcialmente el overhead generado por estas operaciones adicionales.

Los resultados obtenidos evidencian que las consultas analíticas complejas presentan un mejor aprovechamiento del procesamiento paralelo en comparación con operaciones simples de conteo o lectura secuencial de datos.

Ilustración 11 - Tiempo promedio de ejecución Q5 WHERE + GROUP BY



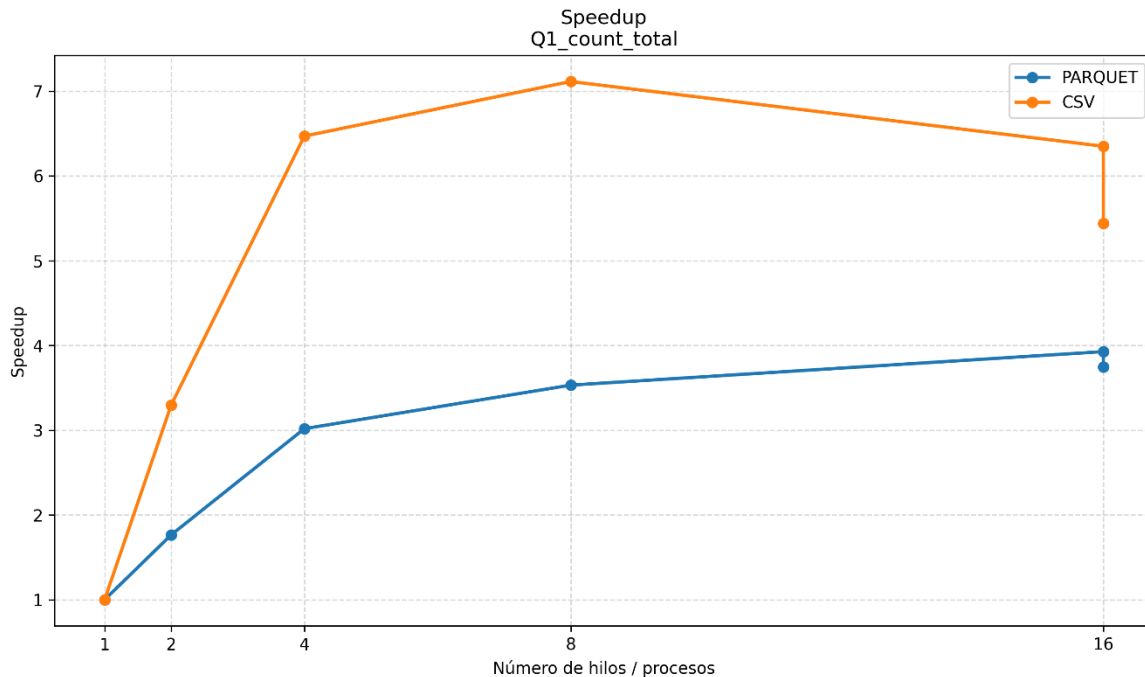
11.5. Resultados de speedup

Con el propósito de evaluar el nivel de aceleración alcanzado mediante procesamiento paralelo, se calculó el speedup para cada una de las consultas analíticas ejecutadas sobre archivos CSV con errores controlados. Esta métrica permitió comparar el tiempo de ejecución obtenido utilizando múltiples hilos frente al tiempo registrado en la ejecución secuencial con un único hilo.

El análisis del speedup permitió identificar el grado de aprovechamiento del paralelismo bajo diferentes niveles de concurrencia y observar el impacto que tienen las tareas de validación y manejo de excepciones sobre la escalabilidad del sistema.

11.5.1. Consulta Q1 – Conteo total de registros

Ilustración 12 - Speedup Q1 COUNT(*)



Análisis: La consulta de conteo total de registros presentó un incremento progresivo del speedup al aumentar el número de hilos de procesamiento. Sin embargo, debido a la baja complejidad computacional de esta operación, la aceleración obtenida fue moderada en comparación con consultas analíticas más complejas.

El mayor crecimiento del speedup se observó entre las configuraciones de 1 y 4 hilos. Posteriormente, el comportamiento comenzó a estabilizarse debido al overhead asociado a sincronización y validación concurrente de datos.

Este comportamiento evidencia que operaciones simples de conteo poseen una capacidad limitada de paralelización efectiva cuando se incorporan mecanismos adicionales de validación y manejo de excepciones.

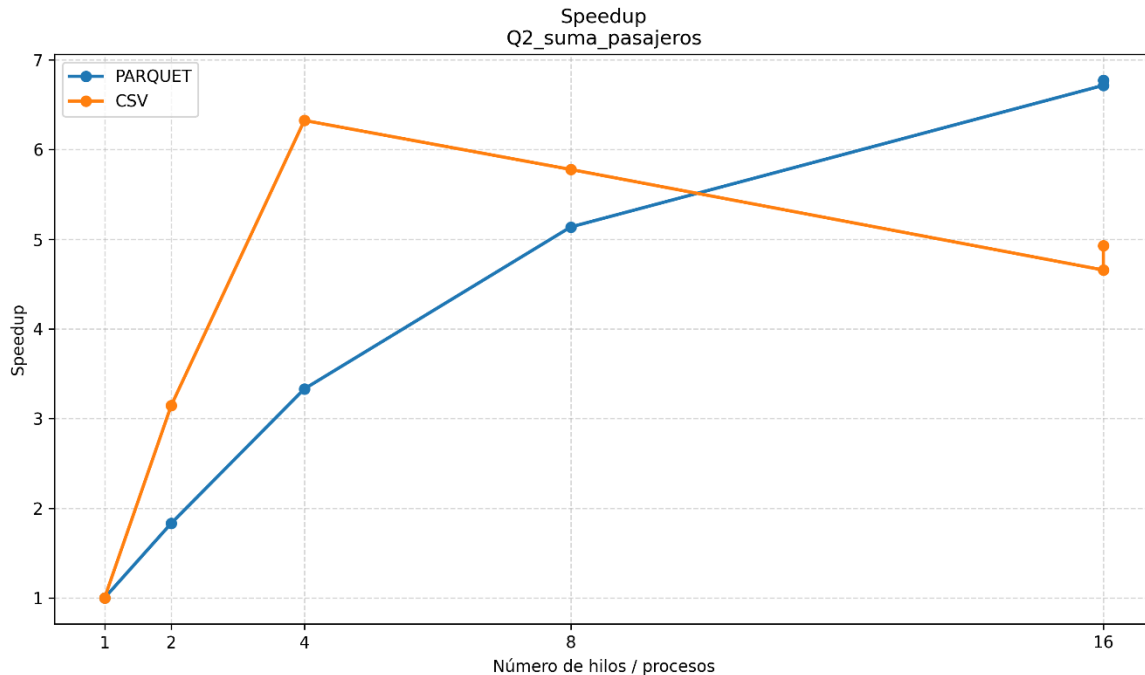
11.5.2. Consulta Q2 – Suma total de pasajeros

Análisis: La operación de suma sobre la columna `passenger_count` mostró un mejor comportamiento escalable en comparación con la consulta de conteo. El incremento en la carga computacional asociado a la agregación numérica permitió distribuir de forma más eficiente el trabajo entre múltiples hilos de procesamiento.

Los resultados mostraron una aceleración considerable hasta configuraciones de 8 hilos, donde el paralelismo logró compensar gran parte del overhead generado por las validaciones implementadas sobre los archivos CSV.

No obstante, al incrementar el nivel de concurrencia hacia 16 hilos, el crecimiento del speedup comenzó a desacelerarse debido al aumento de costos relacionados con sincronización y administración de recursos compartidos.

Ilustración 13 - Speedup Q2 SUM(passenger_count)



Análisis: La operación de suma sobre la columna `passenger_count` mostró un mejor comportamiento escalable en comparación con la consulta de conteo. El incremento en la carga computacional asociado a la agregación numérica permitió distribuir de forma más eficiente el trabajo entre múltiples hilos de procesamiento.

Los resultados mostraron una aceleración considerable hasta configuraciones de 8 hilos, donde el paralelismo logró compensar gran parte del overhead generado por las validaciones implementadas sobre los archivos CSV.

No obstante, al incrementar el nivel de concurrencia hacia 16 hilos, el crecimiento del speedup comenzó a desacelerarse debido al aumento de costos relacionados con sincronización y administración de recursos compartidos.

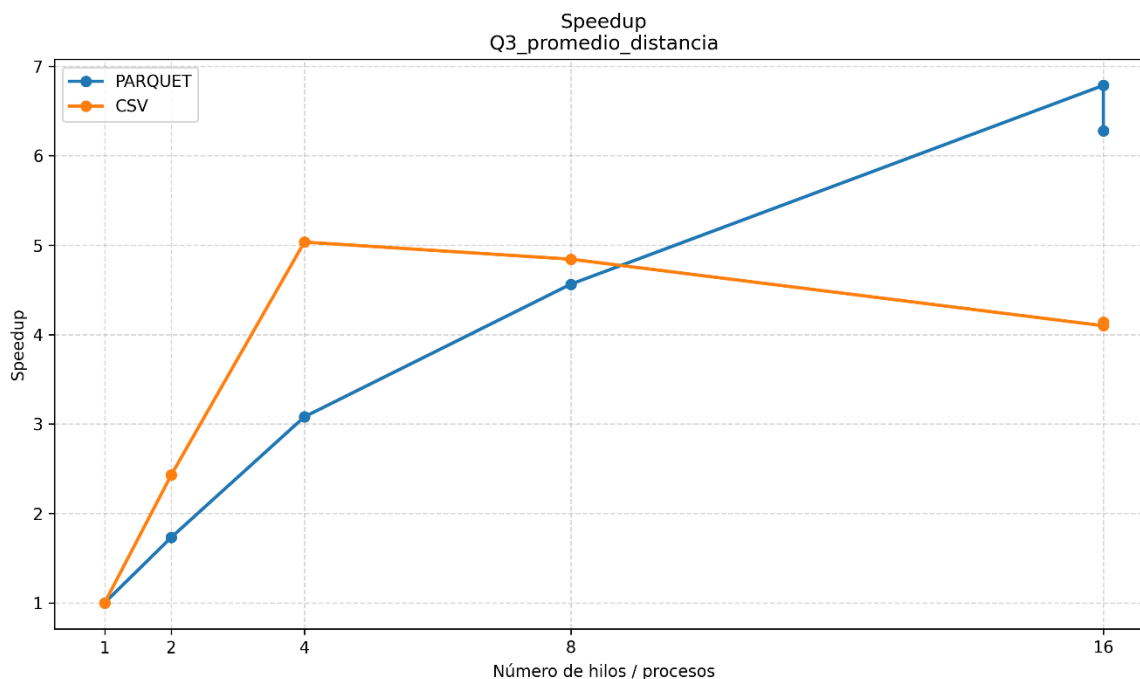
11.5.3. Consulta Q3 – Promedio de distancia recorrida

Análisis: La consulta orientada al cálculo del promedio de distancia recorrida presentó una aceleración progresiva conforme aumentó el número de hilos utilizados. Este comportamiento evidenció que las operaciones estadísticas y de agregación compleja logran beneficiarse significativamente del procesamiento paralelo.

Sin embargo, el procesamiento de esta consulta también implicó tareas adicionales de validación y conversión segura de datos numéricos, lo que introdujo un overhead adicional sobre el sistema.

A pesar de ello, el paralelismo permitió reducir considerablemente el impacto de estas operaciones de validación, alcanzando niveles importantes de aceleración durante configuraciones intermedias de concurrencia.

Ilustración 14 - Speedup Q3 AVG(trip_distance)



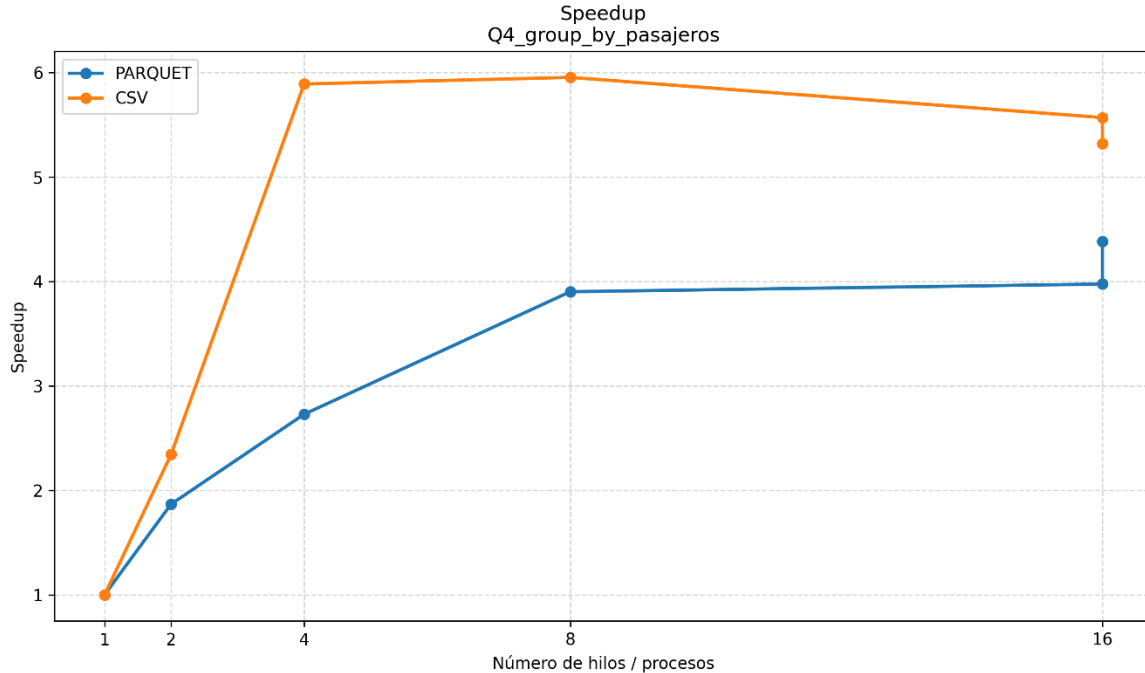
11.5.4. Consulta Q4 – Agrupación por cantidad de pasajeros

Análisis: Las operaciones de agrupamiento mostraron uno de los mejores niveles de speedup dentro del escenario experimental basado en archivos CSV. La consulta GROUP BY permitió distribuir grandes volúmenes de trabajo entre múltiples hilos, favoreciendo el aprovechamiento del paralelismo multinúcleo.

El procesamiento concurrente permitió acelerar significativamente las tareas de agregación parcial y consolidación de resultados. Sin embargo, el crecimiento del speedup comenzó a estabilizarse en configuraciones de alta concurrencia debido al incremento del overhead asociado a sincronización entre hilos y validación concurrente de registros.

Este comportamiento confirma que las operaciones analíticas complejas presentan un mejor potencial de paralelización efectiva en comparación con consultas simples de lectura o conteo.

Ilustración 15 - Speedup Q4 GROUP BY passenger_count



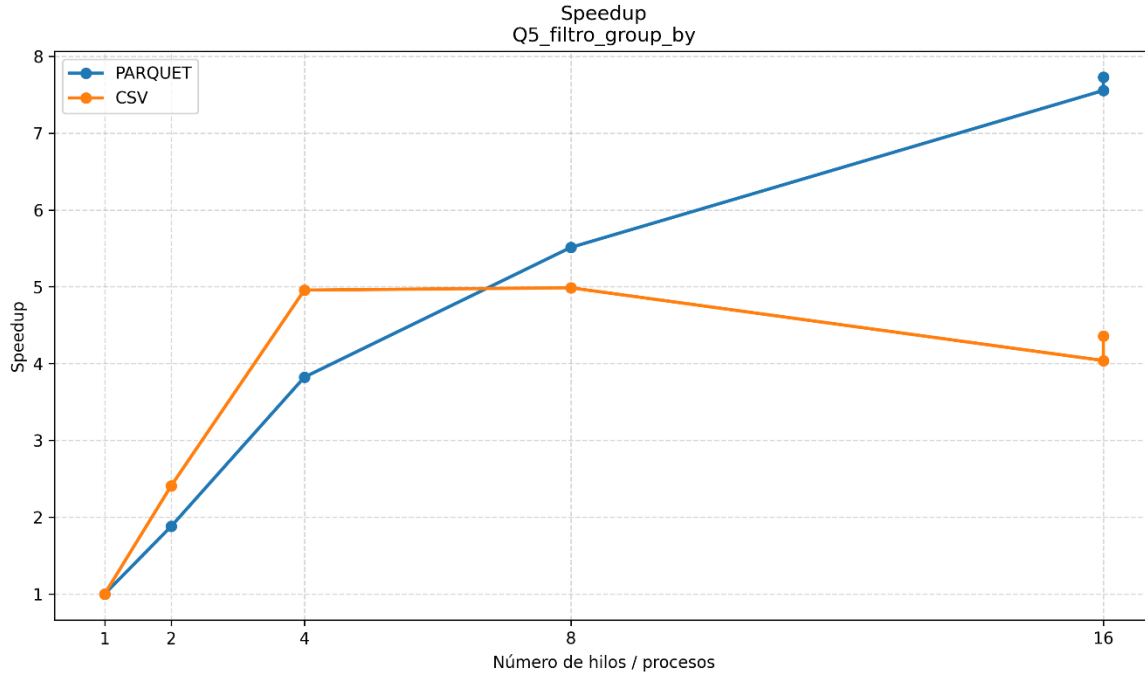
11.5.5. Consulta Q5 – Filtrado y agrupamiento combinado

Análisis: La consulta que combina filtrado y agrupamiento presentó el mayor nivel de aceleración dentro del escenario CSV. La reducción inicial del volumen de datos mediante la cláusula WHERE permitió disminuir parcialmente la carga de procesamiento asociada a las etapas posteriores de agrupamiento y agregación.

Este comportamiento favoreció el aprovechamiento del paralelismo, permitiendo alcanzar los mayores niveles de speedup observados durante el experimento. Los resultados evidenciaron que las consultas analíticas complejas, especialmente aquellas que incorporan filtrado previo de registros, logran aprovechar de manera más eficiente el procesamiento multinúcleo incluso en presencia de validaciones y manejo de excepciones sobre datos inconsistentes.

En términos generales, el análisis del speedup permitió confirmar que el paralelismo mejora significativamente el rendimiento del procesamiento sobre archivos CSV, aunque su efectividad se ve parcialmente limitada por el overhead generado por las tareas de validación y tolerancia a fallos implementadas durante el experimento.

Ilustración 16 - Speedup Q5 WHERE + GROUP BY



11.6. Resultados de eficiencia paralela

Con el propósito de analizar el grado de aprovechamiento de los recursos de procesamiento disponibles, se calculó la eficiencia paralela para cada una de las consultas ejecutadas sobre archivos CSV con errores controlados. Esta métrica permitió evaluar qué tan efectivamente fueron utilizados los hilos de procesamiento a medida que aumentó el nivel de concurrencia del sistema.

La eficiencia paralela fue calculada a partir de la relación entre el speedup obtenido y el número de hilos utilizados durante cada configuración experimental. El análisis de esta métrica permitió identificar pérdidas de rendimiento ocasionadas por overhead de sincronización, validación concurrente de datos y competencia por recursos compartidos.

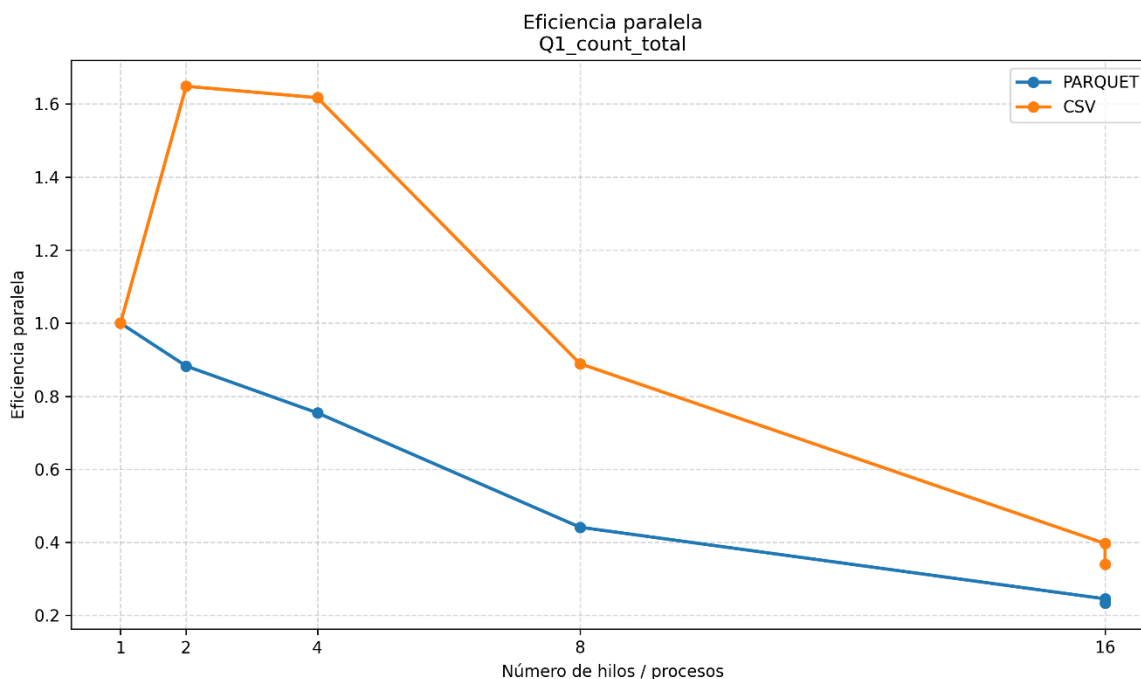
11.6.1. Consulta Q1 – Conteo total de registros

Análisis: La consulta de conteo total de registros presentó una reducción progresiva de la eficiencia paralela conforme aumentó el número de hilos de procesamiento. Este comportamiento es consistente con las limitaciones típicas de paralelización observadas en operaciones de baja complejidad computacional.

Aunque el paralelismo permitió reducir el tiempo total de ejecución, la carga de trabajo distribuable entre hilos fue relativamente limitada, ocasionando pérdidas de eficiencia asociadas al overhead de coordinación y validación concurrente de registros.

En algunas configuraciones intermedias se observaron valores de eficiencia superiores a la unidad, fenómeno asociado a efectos de caché y optimización temporal del acceso a datos, situación ocasionalmente reportada en experimentos de benchmarking paralelo.

Ilustración 17 - Eficiencia paralela Q1 COUNT(*)



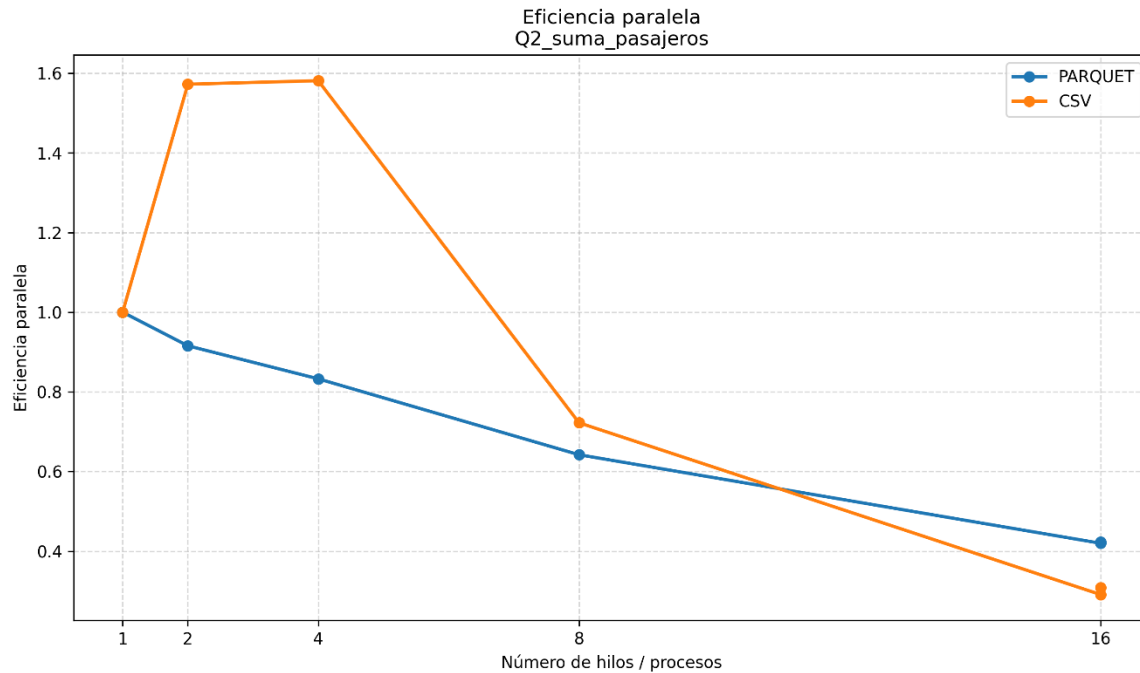
11.6.2. Consulta Q2 – Suma total de pasajeros

Análisis: La operación de suma sobre la columna `passenger_count` mostró un mejor comportamiento escalable en comparación con la consulta de conteo. El incremento en la carga computacional asociado a la agregación numérica permitió distribuir de forma más eficiente el trabajo entre múltiples hilos de procesamiento.

Los resultados mostraron una aceleración considerable hasta configuraciones de 8 hilos, donde el paralelismo logró compensar gran parte del overhead generado por las validaciones implementadas sobre los archivos CSV.

No obstante, al incrementar el nivel de concurrencia hacia 16 hilos, el crecimiento del speedup comenzó a desacelerarse debido al aumento de costos relacionados con sincronización y administración de recursos compartidos.

Ilustración 18 - Eficiencia paralela Q2 SUM(passenger_count)

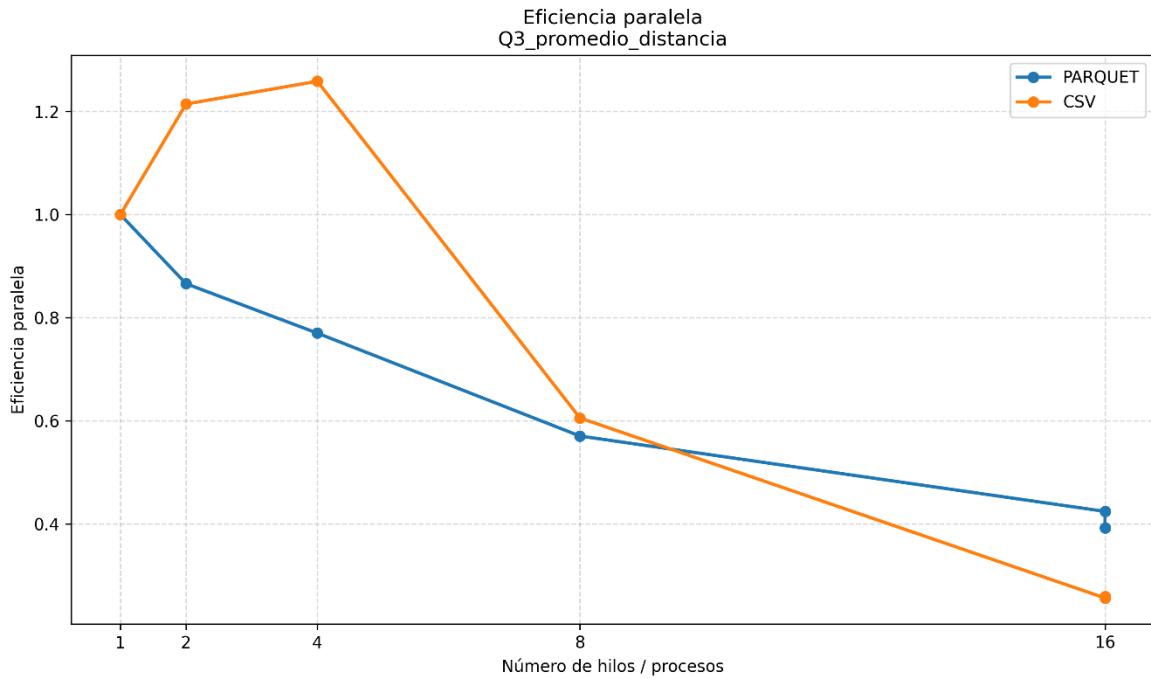


11.6.3. Consulta Q3 – Promedio de distancia recorrida

Análisis: La consulta de promedio mostró un comportamiento escalable favorable durante configuraciones intermedias de paralelismo. La combinación de operaciones estadísticas y validaciones numéricas permitió generar suficiente carga computacional para aprovechar efectivamente múltiples hilos de procesamiento.

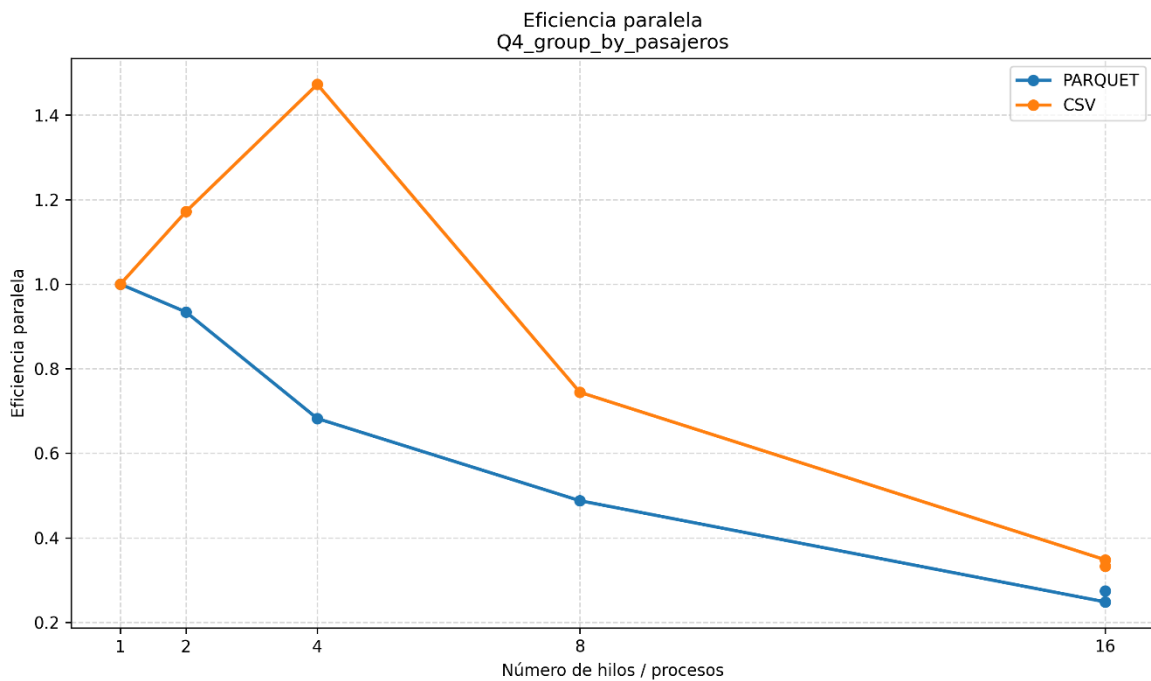
No obstante, el overhead asociado al manejo de excepciones y validación de registros comenzó a impactar progresivamente la eficiencia en configuraciones superiores de concurrencia. A pesar de ello, el sistema logró mantener niveles aceptables de eficiencia paralela incluso bajo escenarios de procesamiento con errores controlados.

Ilustración 19 - Eficiencia paralela Q3 AVG(trip_distance)



11.6.4. Consulta Q4 – Agrupación por cantidad de pasajeros

Ilustración 20 - Eficiencia paralela Q4 GROUP BY passenger_count

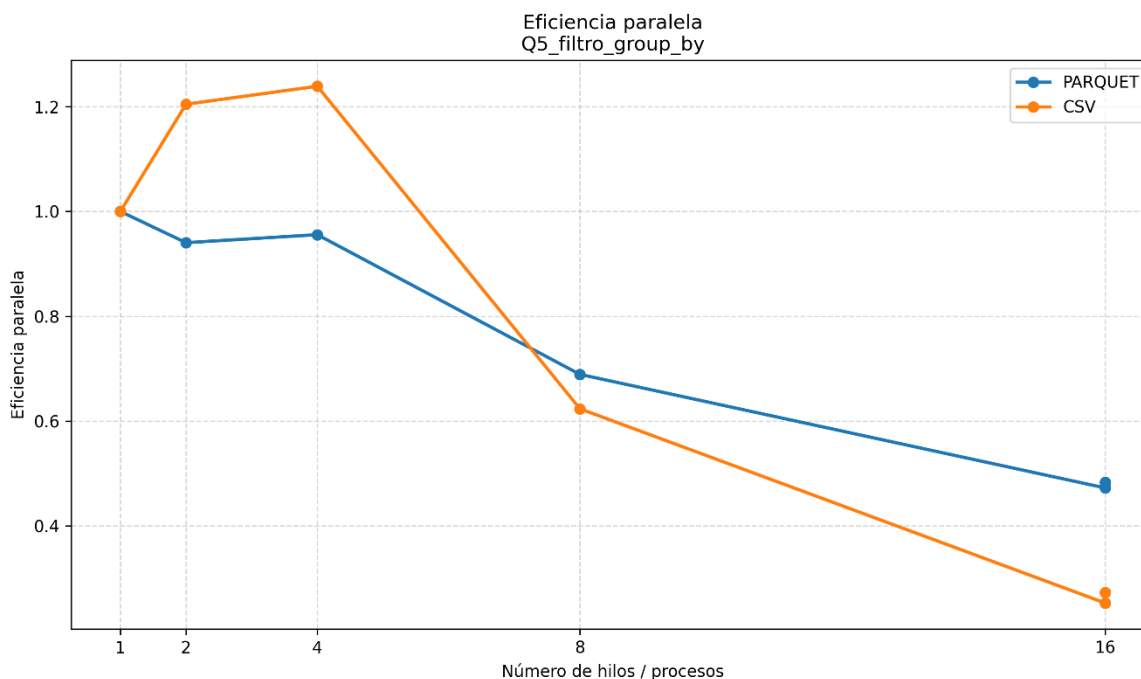


Análisis: Las operaciones de agrupamiento presentaron uno de los mejores comportamientos de eficiencia paralela dentro del escenario experimental basado en archivos CSV. La mayor complejidad computacional de las consultas GROUP BY permitió distribuir de manera más efectiva las tareas entre múltiples hilos de ejecución.

Durante las configuraciones de concurrencia intermedia, el sistema logró mantener niveles elevados de eficiencia, evidenciando un buen aprovechamiento del paralelismo multinúcleo incluso en presencia de validaciones y manejo de errores. Sin embargo, al incrementar el número de hilos hacia configuraciones extremas, comenzaron a evidenciarse pérdidas de eficiencia asociadas al costo de sincronización entre estructuras de agrupamiento concurrentes.

11.6.5. Consulta Q5 – Filtrado y agrupamiento combinado

Ilustración 21 - Eficiencia paralela Q5 WHERE + GROUP BY



Análisis: La consulta que combina filtrado y agrupamiento presentó el mejor comportamiento de eficiencia paralela dentro del escenario CSV. La reducción inicial del volumen de datos mediante el filtrado permitió disminuir parcialmente la carga asociada a las etapas posteriores de agregación y validación.

Este comportamiento favoreció el aprovechamiento del procesamiento multinúcleo, permitiendo mantener niveles elevados de eficiencia durante configuraciones intermedias de concurrencia.

Los resultados evidenciaron que las consultas analíticas complejas logran aprovechar de manera más eficiente el paralelismo, incluso cuando el sistema incorpora mecanismos adicionales de validación de datos y manejo de excepciones.

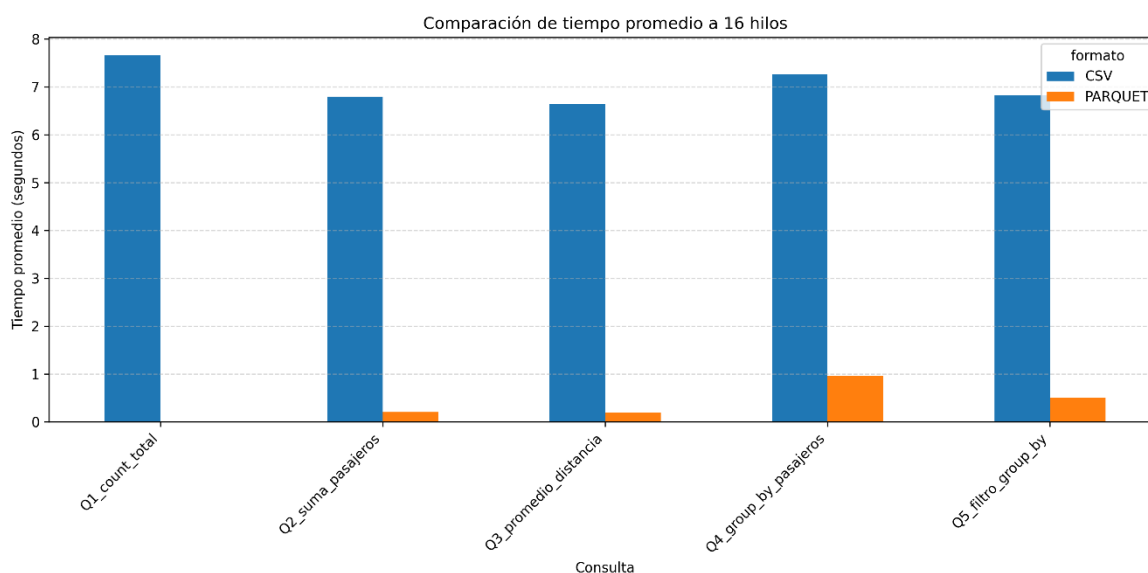
En términos generales, la eficiencia paralela obtenida durante el experimento permitió confirmar que el procesamiento concurrente mejora significativamente el rendimiento del sistema. No obstante, también evidenció que el aumento excesivo del número de hilos introduce overhead adicional asociado a sincronización, administración de recursos y validación concurrente de registros, limitando progresivamente el aprovechamiento efectivo del paralelismo.

11.7. Comparación entre Apache Parquet y CSV

Con el propósito de evaluar las diferencias de rendimiento y escalabilidad entre ambos escenarios experimentales, se realizó una comparación directa entre el procesamiento optimizado utilizando Apache Parquet y el procesamiento basado en archivos CSV con errores controlados.

La comparación se desarrolló utilizando las mismas consultas analíticas y configuraciones de paralelismo, permitiendo analizar el impacto que tienen el formato de almacenamiento, la validación de datos y el manejo de excepciones sobre el comportamiento general del sistema.

11.7.1. Comparación de tiempo de ejecución



Análisis: Los resultados obtenidos evidenciaron diferencias significativas en el tiempo de ejecución entre ambos formatos de procesamiento. El escenario basado en Apache Parquet presentó tiempos considerablemente menores en todas las consultas analíticas evaluadas.

Estas diferencias se deben principalmente a las características de optimización del formato Parquet y al motor analítico DuckDB, el cual aprovecha almacenamiento columnar, lectura selectiva de columnas y mecanismos internos de procesamiento paralelo altamente optimizados. Por otro lado, el procesamiento basado en archivos CSV requirió tareas adicionales de:

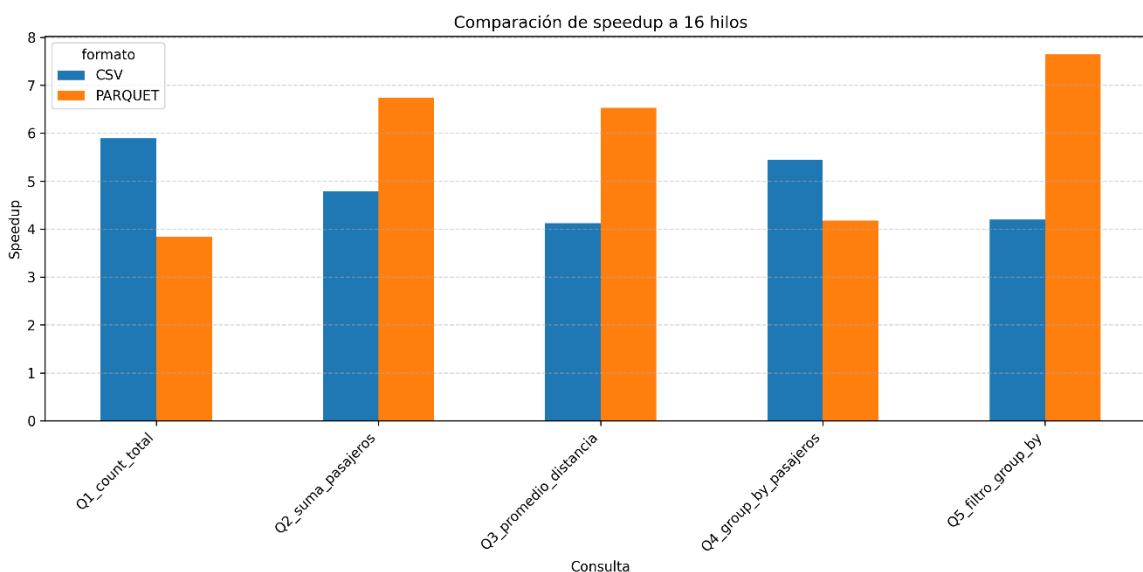
- Lectura secuencial,
- Parsing de texto,
- Validación de datos,
- Manejo de excepciones,
- Verificación de integridad de registros.

Estas operaciones introdujeron un overhead computacional considerable, incrementando el tiempo total de ejecución de las consultas. A pesar de ello, el procesamiento CSV logró mantener tiempos razonables de ejecución considerando que el sistema operó sobre archivos con errores controlados y validaciones concurrentes durante todo el proceso analítico.

Las mayores diferencias de rendimiento se observaron en consultas complejas que involucraron agrupamientos y agregaciones, donde el aprovechamiento de las optimizaciones internas de DuckDB y del almacenamiento columnar resultó especialmente favorable para el escenario Parquet.

11.7.2. Comparación de speedup

Ilustración 22 - Comparación de speedup entre Parquet y CSV utilizando 16 hilos



Análisis: La comparación del speedup mostró que ambos escenarios lograron beneficiarse del procesamiento paralelo multinúcleo. Sin embargo, Apache

Parquet presentó un comportamiento escalable más estable y sostenido al incrementar el número de hilos de procesamiento.

En el caso del procesamiento CSV, el speedup mostró un crecimiento importante durante configuraciones iniciales e intermedias de concurrencia. No obstante, el incremento del número de hilos comenzó a generar pérdidas progresivas de eficiencia debido al overhead asociado a validación concurrente, sincronización y manejo de excepciones.

Las consultas analíticas complejas, particularmente aquellas que combinan filtrado y agrupamiento, alcanzaron los mayores niveles de aceleración tanto en Parquet como en CSV. Este comportamiento evidencia que las cargas analíticas de mayor complejidad presentan un mejor aprovechamiento del paralelismo multinúcleo.

Aun así, el escenario Parquet mantuvo una ventaja clara en términos de escalabilidad debido a la reducción de costos asociados a lectura, parsing y validación de datos.

12. Discusión comparativa general

Los resultados experimentales obtenidos permitieron identificar diferencias importantes entre ambos enfoques de procesamiento. El escenario basado en Apache Parquet demostró ser significativamente más eficiente para tareas analíticas de alto rendimiento, gracias a:

- Su estructura de almacenamiento columnar,
- La reducción del volumen de lectura desde disco,
- Las optimizaciones internas de DuckDB para procesamiento paralelo.

Por otro lado, el procesamiento basado en archivos CSV permitió evaluar un escenario más cercano a condiciones reales de ingeniería de datos, donde la información puede contener inconsistencias, registros inválidos y errores de formato que requieren validación y tolerancia a fallos.

Aunque el escenario CSV presentó tiempos de ejecución mayores y menores niveles de eficiencia paralela, el sistema logró mantener continuidad operativa durante el procesamiento de datos corruptos, evidenciando un comportamiento robusto frente a escenarios reales de calidad de datos. En términos generales, los resultados del experimento permitieron confirmar que:

- El paralelismo multinúcleo mejora significativamente el rendimiento del procesamiento analítico,
- Apache Parquet ofrece ventajas importantes en rendimiento y escalabilidad,
- El manejo de excepciones introduce un costo computacional considerable sobre sistemas de procesamiento masivo de datos.

Estos resultados validan las hipótesis planteadas inicialmente en el proyecto y evidencian la importancia de seleccionar adecuadamente tanto el formato de almacenamiento como las estrategias de validación utilizadas en entornos HPC y Big Data.

13. Discusión general de resultados

Los resultados durante el desarrollo del experimento permitieron analizar el comportamiento del procesamiento paralelo sobre grandes volúmenes de datos bajo dos escenarios diferentes: un entorno optimizado utilizando Apache Parquet y DuckDB, y un entorno basado en archivos CSV con errores controlados y mecanismos de validación implementados en Python.

En términos generales, los resultados evidenciaron que el uso de procesamiento multinúcleo permitió reducir significativamente los tiempos de ejecución en ambos escenarios experimentales. El incremento progresivo del número de hilos produjo mejoras importantes en el rendimiento, especialmente durante las configuraciones intermedias de concurrencia.

Sin embargo, el comportamiento del sistema mostró que el aumento del paralelismo no genera mejoras indefinidas en el rendimiento. A medida que el número de hilos aumentó, comenzaron a evidenciarse efectos asociados a overhead de sincronización, administración de memoria y competencia por recursos compartidos, reduciendo progresivamente la eficiencia paralela del sistema.

Los resultados obtenidos también permitieron identificar diferencias importantes dependiendo del tipo de consulta analítica ejecutada. Las operaciones simples de conteo presentaron menores niveles de aceleración debido a su baja complejidad computacional y limitada capacidad de distribución efectiva entre múltiples hilos.

Por otro lado, las consultas analíticas complejas que incorporaron agrupamientos, filtrados y agregaciones mostraron un mejor aprovechamiento del paralelismo multinúcleo. Particularmente, las operaciones que combinaron filtrado previo de registros y agrupamiento lograron mantener mejores niveles de escalabilidad y eficiencia paralela durante configuraciones de alta concurrencia.

La comparación entre Apache Parquet y CSV evidenció ventajas importantes del almacenamiento columnar y de los motores analíticos optimizados para procesamiento masivo de datos. El escenario basado en Parquet presentó menores tiempos de ejecución y un comportamiento escalable más estable, gracias a:

- La lectura selectiva de columnas,
- La reducción del volumen de datos procesados,
- Las optimizaciones internas de DuckDB para ejecución paralela.

No obstante, el escenario basado en archivos CSV permitió analizar un contexto más cercano a situaciones reales de ingeniería de datos, donde los registros pueden contener inconsistencias, errores de formato y valores inválidos que requieren validación y tolerancia a fallos.

Aunque el manejo de excepciones introdujo un costo computacional considerable sobre el procesamiento, el sistema desarrollado logró mantener continuidad operativa incluso en presencia de registros corruptos o inconsistentes. Este comportamiento evidenció que la robustez y tolerancia a fallos representan componentes fundamentales dentro de escenarios reales de procesamiento masivo de información.

Adicionalmente, el análisis experimental permitió observar que algunas configuraciones intermedias presentaron niveles de eficiencia superiores a la unidad, comportamiento asociado a efectos de caché y optimización temporal del acceso a datos. Este fenómeno ha sido reportado previamente en experimentos de benchmarking paralelo y demuestra la influencia que tienen factores de arquitectura y administración de memoria sobre el rendimiento observado.

En términos generales, los resultados obtenidos permitieron confirmar las hipótesis planteadas inicialmente en el proyecto:

- El paralelismo multinúcleo mejora significativamente el rendimiento del procesamiento analítico,
- Apache Parquet ofrece ventajas importantes en escalabilidad y eficiencia,
- El manejo de errores introduce overhead computacional considerable,
- Las consultas analíticas complejas presentan un mejor aprovechamiento del procesamiento paralelo.

Finalmente, el experimento evidenció la importancia de seleccionar adecuadamente tanto el formato de almacenamiento como las estrategias de validación utilizadas en sistemas HPC y entornos de Big Data, especialmente en escenarios donde el rendimiento y la robustez deben coexistir simultáneamente.

14.Limitaciones del experimento y consideraciones:

Aunque el experimento desarrollado permitió evaluar el comportamiento del procesamiento paralelo sobre grandes volúmenes de datos utilizando diferentes formatos de almacenamiento y mecanismos de validación, es importante reconocer ciertas limitaciones que pueden influir sobre los resultados obtenidos y sobre el alcance de las conclusiones derivadas del estudio.

14.1. Limitaciones del hardware utilizado

Las pruebas experimentales fueron ejecutadas sobre un único equipo con arquitectura multinúcleo, específicamente un procesador AMD Ryzen 9 5900HX con

8 núcleos físicos y 16 hilos lógicos. Aunque esta configuración permitió evaluar el comportamiento del paralelismo en un entorno HPC de escala local, los resultados podrían variar significativamente en sistemas con diferentes capacidades de hardware. Factores como la cantidad de núcleos disponibles, el tamaño de memoria caché, la velocidad de almacenamiento, el ancho de banda de memoria, la arquitectura del procesador puede influir directamente sobre el rendimiento del procesamiento paralelo y sobre el comportamiento de escalabilidad observado durante el experimento.

14.2. Limitaciones del procesamiento distribuido

El presente estudio se enfocó exclusivamente en paralelismo multinúcleo dentro de un único sistema de cómputo. No se evaluaron arquitecturas distribuidas donde múltiples nodos trabajan conjuntamente sobre el procesamiento de grandes volúmenes de información.

Por esta razón, los resultados obtenidos representan únicamente escenarios de paralelismo local (shared-memory) y no permiten extrapolar directamente el comportamiento del sistema hacia entornos distribuidos basados en clusters HPC, Spark o arquitecturas de procesamiento distribuido.

14.3. Diferencias entre motores de procesamiento

La comparación realizada entre Apache Parquet y CSV involucra también diferencias importantes entre los mecanismos de procesamiento utilizados. El escenario basado en Apache Parquet utilizó DuckDB, un motor analítico altamente optimizado para consultas sobre grandes volúmenes de datos y procesamiento paralelo. Por otro lado, el escenario CSV fue implementado mediante rutinas desarrolladas en Python orientadas principalmente a validación, manejo de excepciones y tolerancia a fallos.

Debido a ello, parte de las diferencias de rendimiento observadas pueden estar asociadas no solamente al formato de almacenamiento, sino también al nivel de optimización interno de cada motor de procesamiento.

14.4. Impacto de caché y estado del sistema operativo

Durante la ejecución de los experimentos, factores externos relacionados con el sistema operativo y el estado de memoria pudieron influir parcialmente sobre los tiempos registrados. Aspectos como caché de disco, procesos en segundo plano, administración dinámica de energía, planificación de hilos del sistema operativo, y optimizaciones automáticas del procesador pueden introducir pequeñas variaciones entre ejecuciones consecutivas.

Aunque se realizaron múltiples ejecuciones y pruebas de calentamiento (warmup) para reducir estas variaciones, no es posible eliminarlas completamente en entornos de ejecución reales.

14.5. Limitaciones relacionadas con los datos generados

Los archivos CSV utilizados en el experimento fueron generados artificialmente a partir del dataset original mediante la inserción controlada de errores e inconsistencias.

Aunque esta estrategia permitió simular escenarios reales de calidad de datos, los patrones de errores generados pueden diferir de aquellos encontrados en sistemas reales de producción, donde las inconsistencias suelen presentar comportamientos más heterogéneos y menos predecibles.

Por esta razón, los resultados obtenidos deben interpretarse como una aproximación experimental orientada a evaluar robustez y tolerancia a fallos bajo condiciones controladas.

14.6. Sobrecarga del paralelismo

Los resultados experimentales evidenciaron que el incremento del número de hilos no siempre genera mejoras proporcionales en el rendimiento del sistema.

A partir de configuraciones de alta concurrencia comenzaron a observarse pérdidas progresivas de eficiencia ocasionadas por sincronización entre hilos, competencia por memoria, coordinación de tareas concurrentes y overhead de validación y manejo de excepciones.

Este comportamiento representa una limitación inherente a los sistemas de procesamiento paralelo y demuestra que el paralelismo posee límites prácticos asociados a la arquitectura del hardware y al tipo de carga analítica ejecutada.

14.7. Alcance experimental del estudio

El experimento desarrollado se enfocó principalmente en operaciones analíticas orientadas a:

- Conteo,
- Agregaciones,
- Filtrados,
- Agrupamientos,
- Validación de datos.

No se evaluaron escenarios más complejos relacionados con:

- Aprendizaje automático.

- procesamiento distribuido.
- consultas geoespaciales.
- streaming de datos.
- Procesamiento en tiempo real.

Por esta razón, los resultados representan específicamente el comportamiento del procesamiento analítico paralelo sobre grandes volúmenes de datos tabulares estructurados y semi-estructurados dentro de un entorno HPC local.

15.Conclusiones

El desarrollo del presente proyecto permitió evaluar el impacto del paralelismo multinúcleo sobre el procesamiento de grandes volúmenes de datos utilizando diferentes formatos de almacenamiento y estrategias de validación en un entorno de Computación de Alto Desempeño (HPC).

Los resultados experimentales obtenidos evidenciaron que el uso de procesamiento paralelo permitió reducir significativamente los tiempos de ejecución en las operaciones analíticas ejecutadas tanto sobre archivos Apache Parquet como sobre archivos CSV con errores controlados. El incremento del número de hilos de procesamiento produjo mejoras importantes de rendimiento, especialmente en consultas analíticas complejas que involucraron operaciones de agrupamiento, filtrado y agregación.

El escenario experimental basado en Apache Parquet presentó el mejor comportamiento en términos de rendimiento y escalabilidad. Las características de almacenamiento columnar, lectura selectiva de columnas y optimizaciones internas de DuckDB permitieron reducir considerablemente los tiempos de ejecución y mantener un comportamiento escalable más estable frente al incremento del paralelismo.

Por otro lado, el procesamiento basado en archivos CSV permitió evaluar escenarios más cercanos a contextos reales de ingeniería de datos, donde los registros pueden contener inconsistencias, errores de formato y valores inválidos que requieren validación y manejo de excepciones durante la ejecución.

Aunque el procesamiento CSV presentó tiempos de ejecución superiores y menores niveles de eficiencia paralela, el sistema implementado logró mantener continuidad operativa incluso en presencia de registros corruptos o inconsistentes. Este comportamiento permitió demostrar que los mecanismos de validación y tolerancia a fallos constituyen componentes fundamentales dentro de sistemas reales de procesamiento masivo de información.

Los resultados también evidenciaron que las consultas analíticas complejas presentan un mejor aprovechamiento del paralelismo en comparación con operaciones simples de conteo o lectura secuencial. Particularmente, las operaciones que combinaron filtrado y agrupamiento alcanzaron los mayores niveles de speedup y eficiencia paralela durante el experimento.

Adicionalmente, el análisis experimental permitió identificar que el incremento del número de hilos no genera mejoras indefinidas en el rendimiento del sistema. A partir de configuraciones de alta concurrencia comenzaron a evidenciarse efectos asociados a overhead de sincronización, competencia por memoria y validación concurrente de datos, reduciendo progresivamente la eficiencia del procesamiento paralelo.

El experimento también permitió confirmar que el manejo de excepciones y validación de datos introduce un costo computacional considerable sobre el procesamiento de archivos CSV. Sin embargo, este overhead representa un componente necesario en escenarios reales donde la calidad de los datos no puede garantizarse completamente.

En términos generales, las hipótesis planteadas inicialmente en el proyecto fueron confirmadas:

- El paralelismo multinúcleo mejora significativamente el rendimiento del procesamiento analítico,
- Apache Parquet ofrece ventajas importantes en rendimiento y escalabilidad,
- El manejo de errores introduce overhead computacional adicional,
- Las consultas analíticas complejas presentan un mejor aprovechamiento del procesamiento paralelo.

Finalmente, el proyecto permitió demostrar la importancia de seleccionar adecuadamente tanto el formato de almacenamiento como las estrategias de procesamiento y validación utilizadas en aplicaciones de Big Data y entornos HPC, especialmente en escenarios donde deben coexistir simultáneamente rendimiento, escalabilidad y robustez frente a errores de datos.

16. Trabajo futuro y propuestas de ampliación

A partir de los resultados obtenidos durante el desarrollo del presente proyecto, se identifican diversas líneas de trabajo futuro orientadas a ampliar el alcance experimental y profundizar el análisis del procesamiento paralelo sobre grandes volúmenes de datos en entornos HPC y Big Data.

Una de las principales propuestas de ampliación consiste en extender el experimento hacia arquitecturas de procesamiento distribuido utilizando múltiples nodos de ejecución. El presente estudio se enfocó en paralelismo multinúcleo dentro de un único sistema de cómputo (shared-memory); sin embargo, en escenarios reales de Big Data es común utilizar clusters distribuidos donde múltiples máquinas trabajan conjuntamente sobre datasets de gran escala. La implementación de una arquitectura distribuida permitiría evaluar:

- Escalabilidad horizontal,
- Distribución de carga entre nodos,
- Comunicación y sincronización distribuida,
- Tolerancia a fallos entre múltiples máquinas,
- Comportamiento del procesamiento analítico frente a volúmenes de datos considerablemente mayores.

Bajo este enfoque, podrían incorporarse tecnologías orientadas a procesamiento distribuido como:

- Apache Spark
- Dask
- Ray
- MPI para Python
- Plataformas de cómputo distribuido orientadas a HPC.

Adicionalmente, futuras investigaciones podrían ampliar el tamaño del dataset utilizado, incorporando periodos históricos más extensos o combinando múltiples fuentes de datos relacionadas con movilidad urbana, sensores IoT o sistemas de monitoreo en tiempo real.

Otra línea de ampliación importante consiste en optimizar los mecanismos de validación y manejo de excepciones implementados sobre archivos CSV. Aunque el presente proyecto permitió demostrar la robustez del sistema frente a registros inconsistentes, también evidenció el alto costo computacional asociado a las tareas de validación concurrente. En este sentido, futuros trabajos podrían explorar:

- Validación distribuida de datos,
- Procesamiento batch de excepciones,
- Pipelines de limpieza previa,
- Estructuras de validación vectorizada,
- Aceleración mediante bibliotecas optimizadas para procesamiento columnar.

También se propone evaluar el comportamiento del procesamiento paralelo utilizando aceleración por GPU, aprovechando arquitecturas CUDA o bibliotecas especializadas para análisis masivo de datos. Este enfoque permitiría comparar el rendimiento entre procesamiento CPU multinúcleo y procesamiento heterogéneo CPU-GPU. De igual manera, futuras investigaciones podrían incorporar escenarios analíticos de mayor complejidad relacionados con:

- Aprendizaje automático,
- Procesamiento geoespacial,
- Análisis temporal avanzado,
- Minería de datos,
- Procesamiento de streams en tiempo real.

Otra posible ampliación del proyecto consiste en desarrollar una interfaz gráfica o plataforma interactiva que permita ejecutar benchmarks dinámicos, controlar configuraciones de paralelismo y visualizar métricas de rendimiento en tiempo real. Esto facilitaría la utilización del sistema como herramienta académica y de experimentación en cursos relacionados con HPC, Big Data y procesamiento paralelo.

Finalmente, el presente trabajo abre la posibilidad de continuar investigando la relación existente entre rendimiento, robustez y calidad de datos dentro de sistemas de procesamiento masivo de información, especialmente en entornos donde el paralelismo y la tolerancia a fallos deben coexistir simultáneamente.

17. Bibliografía

1. DuckDB Official Documentation. <https://duckdb.org/docs/>
2. Apache Parquet Documentation. <https://parquet.apache.org/docs/>
3. NYC Taxi & Limousine Commission – Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
4. Stonebraker, M., & Çetintemel, U. (2005). “One Size Fits All”: An Idea Whose Time Has Come and Gone. Proceedings of the 21st International Conference on Data Engineering (ICDE). IEEE.
5. Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified Data Processing on Large Clusters. Communications of the ACM, 51(1), 107–113.
6. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media.
7. Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). Introduction to Parallel Computing (2nd ed.). Addison-Wesley.
8. Wilkinson, B., & Allen, M. (2005). Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers (2nd ed.). Pearson.
9. Rauber, T., & Rünger, G. (2013). Parallel Programming for Multicore and Cluster Systems. Springer.
10. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
11. Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
12. McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.
13. Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster Computing with Working Sets. Proceedings of the 2nd USENIX Conference on Hot Topics in Cloud Computing.
14. Gropp, W., Lusk, E., & Skjellum, A. (2014). Using MPI: Portable Parallel Programming with the Message Passing Interface (3rd ed.). MIT Press.
15. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

16. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
17. Han, J., Pei, J., & Kamber, M. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.
18. Apache Software Foundation. Apache Spark Documentation. <https://spark.apache.org/docs/latest/>
19. Dask Development Team. Dask Documentation. <https://docs.dask.org/en/stable/>
20. NVIDIA Corporation. CUDA Toolkit Documentation. <https://docs.nvidia.com/cuda/>